

COMP2017 Final Exam Revision Notes

Xiyuan Pan

May 2026

Contents

1	Week 1: Introduction to C	8
1.1	Core idea	8
1.2	Shape of a C program	8
1.3	Compiling and running	8
1.4	Functions and declarations	9
1.5	Control flow	9
1.6	Basic input and output	10
1.7	Standard streams	11
1.8	Manual pages	11
1.9	Tutorial Extractions	11
1.9.1	Pointer value, address, and dereference	11
1.10	Full-depth expansion pass	12
1.10.1	Teaching-density checklist	12
1.10.2	Expanded API cards	13
1.10.3	Tutorial extractions with full reasoning	14
1.10.4	Pointer print trace	14
1.10.5	Compile-run-debug loop	15
1.10.6	Exam-readiness prompts	15
1.10.7	Depth takeaway	16
2	Week 2: Memory & Pointers	16
2.1	Core idea	16
2.2	Representation and types	16
2.3	Pointers	16
2.4	Arrays versus pointers	17
2.5	Pointer safety rules	17
2.6	Tutorial Extractions	17

2.6.1	Arrays, pointer decay, and sizeof	17
2.6.2	Safer input with fgets	18
2.7	Full-depth expansion pass	19
2.7.1	Teaching-density checklist	19
2.7.2	Expanded API cards	19
2.7.3	Tutorial extractions with full reasoning	21
2.7.4	Array decay and length passing	21
2.7.5	Pointer alias update	21
2.7.6	Exam-readiness prompts	22
2.7.7	Depth takeaway	22
3	Week 3: String Manipulation	22
3.1	Core idea	22
3.2	String storage	22
3.3	Length and bounded length	23
3.4	Copying strings	23
3.5	Comparing strings	23
3.6	Character stream transformation	24
3.7	Bitwise operations and math tooling	25
3.8	Tutorial Extractions	25
3.8.1	Implementing strlen	25
3.8.2	Bounded string comparison	25
3.9	Full-depth expansion pass	26
3.9.1	Teaching-density checklist	26
3.9.2	Expanded API cards	26
3.9.3	Tutorial extractions with full reasoning	28
3.9.4	Implement strlen carefully	28
3.9.5	Run-length encoding boundary	28
3.9.6	Exam-readiness prompts	29
3.9.7	Depth takeaway	29
4	Week 4: Enums, Structs, Unions, Files	29
4.1	Core idea	29
4.2	Enums	30
4.3	Structs and typedef	30
4.4	Standard library structs	31
4.5	Padding and alignment	31

4.6	Unions and tagged unions	31
4.7	Bitfields and bit operations	32
4.8	Streams and FILE *	32
4.9	Binary files and struct layout	34
4.10	Tutorial Extractions	35
4.10.1	Tagged union as a reusable design pattern	35
4.10.2	Text file read pattern	35
4.11	Full-depth expansion pass	36
4.11.1	Teaching-density checklist	36
4.11.2	Expanded API cards	36
4.11.3	Tutorial extractions with full reasoning	38
4.11.4	Tagged union invariant	38
4.11.5	feof loop correction	38
4.11.6	Exam-readiness prompts	39
4.11.7	Depth takeaway	39
5	Week 5: Dynamic Memory and Linked Lists	39
5.1	Core idea	39
5.2	Stack, static storage, and heap	39
5.3	Allocation APIs	40
5.4	Memory debugging tools	41
5.5	Linked lists	41
5.6	Double pointer list manipulation	42
5.7	Tutorial Extractions	42
5.7.1	Dynamic array growth	42
5.7.2	Storage duration	43
5.8	Full-depth expansion pass	44
5.8.1	Teaching-density checklist	44
5.8.2	Expanded API cards	44
5.8.3	Tutorial extractions with full reasoning	45
5.8.4	Delete from a linked list	45
5.8.5	Dynamic array append	46
5.8.6	Exam-readiness prompts	47
5.8.7	Depth takeaway	47
6	Week 6: Function Pointers, Signals, errno, Low-level I/O	47
6.1	Core idea	47

6.2	Function pointers	47
6.3	Signals	48
6.4	errno and diagnostics	49
6.5	Low-level file descriptors	50
6.6	Non-blocking file descriptor flags	52
6.7	Tutorial Extractions	52
6.7.1	qsort comparator	52
6.7.2	Non-blocking stdin pulse	53
6.8	Full-depth expansion pass	53
6.8.1	Teaching-density checklist	53
6.8.2	Expanded API cards	54
6.8.3	Tutorial extractions with full reasoning	55
6.8.4	qsort struct comparator	55
6.8.5	Robust copy loop	56
6.8.6	Exam-readiness prompts	57
6.8.7	Depth takeaway	57
7	Week 7: Compilation, Linking, Preprocessor, Type Safety	57
7.1	Core idea	57
7.2	Compilation stages	57
7.3	Headers, declarations, and definitions	58
7.4	Preprocessor	58
7.5	Linkage: extern and static	58
7.6	const and restrict	59
7.7	Object inspection tools	59
7.8	Tutorial Extractions	59
7.8.1	Multi-file linkage	59
7.8.2	Tree DFS with dynamic children	60
7.9	Full-depth expansion pass	60
7.9.1	Teaching-density checklist	61
7.9.2	Expanded API cards	61
7.9.3	Tutorial extractions with full reasoning	63
7.9.4	Multi-file undefined reference	63
7.9.5	Macro expansion trap	64
7.9.6	Const and restrict API design	64
7.9.7	Exam-readiness prompts	65
7.9.8	Depth takeaway	65

8	Week 8: Processes, fork, exec, wait, posix_spawn	65
8.1	Core idea	65
8.2	Process memory image	65
8.3	fork	65
8.4	exec family	66
8.5	Waiting and exit status	67
8.6	Zombies and fork bombs	68
8.7	posix_spawn	68
8.8	Tutorial Extractions	69
8.8.1	Fork output counting	69
8.8.2	Mini-shell pattern	69
8.9	Full-depth expansion pass	70
8.9.1	Teaching-density checklist	70
8.9.2	Expanded API cards	70
8.9.3	Tutorial extractions with full reasoning	72
8.9.4	Fork output count	72
8.9.5	Mini-shell command	73
8.9.6	Exam-readiness prompts	74
8.9.7	Depth takeaway	74
9	Week 9: IPC, Pipes, Redirection, popen/pclose	74
9.1	Core idea	74
9.2	pipe	74
9.3	Pipe EOF and unused ends	75
9.4	dup, dup2, fdopen, fileno	75
9.5	popen and pclose	76
9.6	posix_spawn file actions	77
9.7	Tutorial Extractions	77
9.7.1	Parent-child request/response	77
9.7.2	Mini-shell pipeline	78
9.8	Full-depth expansion pass	78
9.8.1	Teaching-density checklist	78
9.8.2	Expanded API cards	79
9.8.3	Tutorial extractions with full reasoning	81
9.8.4	Parent-child two-pipe exchange	81
9.8.5	Pipeline redirection	82
9.8.6	Readiness API comparison	83

9.8.7	Exam-readiness prompts	83
9.8.8	Depth takeaway	83
9.9	Extended IPC: mmap, select, poll, epoll, Non-blocking I/O	84
9.10	Core idea	84
9.11	mmap and munmap	84
9.12	select	85
9.13	poll	86
9.14	epoll	86
9.15	Blocking versus non-blocking design	87
9.16	Tutorial Extractions	87
9.16.1	mmap word scanning	87
9.16.2	Monitoring many children	87
10	Week 10: Parallelism and POSIX Threads	88
10.1	Full-depth expansion pass	88
10.1.1	Teaching-density checklist	88
10.1.2	Expanded API cards	88
10.1.3	Tutorial extractions with full reasoning	90
10.1.4	Safe thread arguments	90
10.1.5	Parallel sum without shared writes	91
10.1.6	Amdahl speedup limit	92
10.1.7	Exam-readiness prompts	92
10.1.8	Depth takeaway	92
11	Week 11: Thread Synchronisation and Further Synchronisation	93
11.1	Core idea	93
11.2	Parallelism motivation	93
11.3	POSIX threads basics	93
11.4	Thread argument passing	94
11.5	Race conditions and mutexes	94
11.6	Semaphores	95
11.7	Condition variables	95
11.8	Barriers	96
11.9	Deadlock, livelock, starvation	97
11.10	Amdahl's Law	97
11.11	Tutorial Extractions	98
11.11.1	Thread argument passing	98

11.11.2	Producer-consumer condition	98
11.11.3	Amdahl calculation	98
11.12	Full-depth expansion pass	98
11.12.1	Teaching-density checklist	98
11.12.2	Expanded API cards	99
11.12.3	Tutorial extractions with full reasoning	101
11.12.4	Race on counter	101
11.12.5	Producer-consumer condition	102
11.12.6	Deadlock by lock order	103
11.12.7	Barrier phase split	103
11.12.8	Exam-readiness prompts	104
11.12.9	Depth takeaway	104
12	Final Revision Patterns and Exam Traps	104
12.1	Patterns to recognise	104
12.2	High-value exam traps	105
12.3	Function / Macro / Type / Tooling Coverage Checklist	105

1 Week 1: Introduction to C

1.1 Core idea

C is a procedural systems programming language. It gives the programmer direct control over memory, program layout, compilation, and interaction with the operating system. That control is powerful, but it also means mistakes are often real memory mistakes rather than friendly runtime exceptions.

The most important mental shift from Java-style programming is that C does not protect you from the machine. A variable is storage, an array is contiguous memory, a string is a convention, and a pointer is an address. The compiler will translate the program, but it will not automatically manage object lifetimes or check every boundary for you.

1.2 Shape of a C program

A C program is made from source files, headers, declarations, definitions, and one entry point called `main`. The compiler sees text after preprocessing, then produces object code, and the linker combines object files and libraries into a final executable.

```
1 #include <stdio.h>
2
3 int main(int argc, char **argv) {
4     printf("Hello World!\n");
5     return 0;
6 }
```

Listing 1: Minimal C program

Important lines:

- `##include <stdio.h>` makes declarations such as `printf` visible.
- `argc` is the number of command-line arguments.
- `argv` points to the argument strings.
- `return 0;` conventionally means successful program termination.

1.3 Compiling and running

```
1 gcc hello.c -o hello
2 ./hello
3
4 gcc hello.c
5 ./a.out
```

Listing 2: Basic compile and run workflow

`gcc`

- **Context/header:** Shell command, man section 1.
- **Form:** `gcc source.c -o program`
- **Purpose:** Compile and link a C source file into an executable.
- **Result/trap:** If `-o` is omitted, the default executable is often `a.out`. Compiler diagnostics are part of the workflow, not optional noise.

-Wall, -Wextra, -Werror

- **Context/header:** Compiler warning flags.
- **Form:** `gcc -Wall -Wextra -Werror main.c -o main`
- **Purpose:** Enable many useful warnings, extra warnings, and treat warnings as errors.
- **Result/trap:** Excellent for COMP2017 work because undefined behaviour often begins as a warning. `-Werror` can stop builds on warnings, so fix the warning rather than hiding it.

-g, -O0

- **Context/header:** Debug build flags.
- **Form:** `gcc -g -O0 main.c -o main`
- **Purpose:** Include debug symbols and avoid optimisation that makes debugging harder.
- **Result/trap:** Use these with `gdb`, `AddressSanitizer`, and `Valgrind`. Optimised code can reorder or remove variables, which makes beginner debugging confusing.

1.4 Functions and declarations

A function has a return type, a name, parameters with types, and a body. A declaration tells the compiler a function exists; a definition provides the body.

```
1 int factorial(int n) {  
2     if (n <= 1) {  
3         return 1;  
4     }  
5     return n * factorial(n - 1);  
6 }  
7  
8 int foo(float, char);  
9 extern int foo(float, char);  
10 void bar(void);
```

Listing 3: Definition and declarations

Use `void` as the return type when a function returns no value. Use `void` in the parameter list when it takes no parameters. A declaration without a body ends with a semicolon.

1.5 Control flow

C has familiar control structures: `if`, `else`, `while`, `do while`, `for`, and `switch`. The important exam traps are semicolons, missing braces, fall-through in `switch`, and integer truth values.

```
1 for (int i = 0; i < 100; i++) {  
2     counter[i] = i;  
3 }  
4  
5 switch (choice) {  
6 case 1:  
7     puts("one");  
8     break;  
9 default:  
10    puts("other");  
11    break;  
12 }
```

1.6 Basic input and output

The early course uses both character-at-a-time I/O and formatted I/O. These are standard library calls, normally declared in `<stdio.h>`.

`printf`

- **Context/header:** `##include <stdio.h>`, library call, man section 3.
- **Form:** `int printf(const char *format, ...);`
- **Purpose:** Print formatted output to `stdout`. Format specifiers such as `%d`, `%s`, `%p`, and `%zu` decide how later arguments are interpreted.
- **Result/trap:** Returns the number of characters printed, or a negative value on output error. The format string and argument types must match; `%p` expects a `void *`.

`scanf`

- **Context/header:** `##include <stdio.h>`, man section 3.
- **Form:** `int scanf(const char *format, ...);`
- **Purpose:** Read formatted input from `stdin`. Most conversion arguments must be addresses because `scanf` writes into caller variables.
- **Result/trap:** Returns the number of items successfully assigned. Always check it. For strings, include a width such as `%31s`; otherwise input can overflow the destination.

`getchar`, `putchar`

- **Context/header:** `##include <stdio.h>`.
- **Form:** `int getchar(void); int putchar(int c);`
- **Purpose:** Read and write one character through standard input/output.
- **Result/trap:** Use `int`, not `char`, for the result of `getchar` so EOF can be represented.

```

1  #include <stdio.h>
2
3  int main(void) {
4      int fahrenheit;
5
6      printf("Please enter a fahrenheit temperature: ");
7      if (scanf("%d", &fahrenheit) != 1) {
8          fprintf(stderr, "invalid input\n");
9          return 1;
10     }
11
12     printf("%d fahrenheit is %d centigrade\n",
13           fahrenheit, (fahrenheit - 32) * 5 / 9);
14     return 0;
15 }
```

Listing 5: Basic input example

1.7 Standard streams

`stdin`, `stdout`, `stderr`

- **Context/header:** `##include <stdio.h>`.
- **Form:** `FILE *stdin; FILE *stdout; FILE *stderr;`
- **Purpose:** Standard input, standard output, and standard error streams available when a process starts.
- **Result/trap:** They are `FILE *` streams, not file descriptor integers. At the low-level descriptor layer, the corresponding descriptors are normally 0, 1, and 2.

`fprintf`

- **Context/header:** `##include <stdio.h>`.
- **Form:** `int fprintf(FILE *stream, const char *format, ...);`
- **Purpose:** Formatted output to a chosen stream, commonly `stderr` for diagnostics.
- **Result/trap:** Same format-string traps as `printf`. Use it for errors instead of mixing diagnostics into normal output.

1.8 Manual pages

The Unix manual is a required course tool.

```
1 man man
2 man 1 gcc
3 man 2 open
4 man 3 printf
```

Listing 6: Manual page sections

- `man 1`: shell commands and executable programs, such as `gcc`.
- `man 2`: system calls, such as `open`, `read`, `fork`, and `pipe`.
- `man 3`: C library functions, such as `printf`, `malloc`, `qsort`, and `popen`.

Common pitfalls

- Forgetting `&` in `scanf("%d", &x)`.
- Using the wrong format specifier, especially for pointers and `size_t`.
- Treating warnings as harmless.
- Forgetting that command-line arguments are strings, not already-parsed integers.
- Confusing standard streams with low-level file descriptors.

1.9 Tutorial Extractions

1.9.1 Pointer value, address, and dereference

This pattern appears very early because it is the foundation for later arrays, linked lists, pipes, and thread argument passing.

```

1 int x = 10;
2 int *p = &x;
3 int **pp = &p;
4
5 printf("%d\n", x);           /* value: 10 */
6 printf("%p\n", (void *)&x); /* address of x */
7 printf("%p\n", (void *)p);  /* same address as &x */
8 printf("%d\n", *p);         /* value stored at that address */
9 printf("%p\n", (void *)pp); /* address of p */
10 printf("%p\n", (void *)*pp);
11 printf("%d\n", **pp);

```

Listing 7: Distinguishing value, address, and dereference

Reasoning:

- `x` means the stored integer.
- `&x` means the address of that storage.
- `p` stores an address.
- `*p` follows the address and accesses the object there.
- `pp` points to a pointer, so `**pp` takes two dereference steps.

Week takeaway

- C programs are files, functions, headers, and one `main`.
- `gcc`, warnings, debug flags, and manual pages are core workflow skills.
- `printf`, `scanf`, `getchar`, and `putchar` are useful only when their return values and types are respected.
- Address/value/dereference thinking starts here and continues through the whole course.

1.10 Full-depth expansion pass

The original Week 1 density is about forming habits, not merely remembering syntax. Read every short C program as a storage story: which objects exist, which names refer to them, which calls can fail, and which values cross the boundary between the shell, the C runtime, and standard streams.

1.10.1 Teaching-density checklist

Read this week using the original notes standard: each item below should be explainable in words, recognisable in code, and connectable to a failure mode. If an item feels like a vocabulary word only, rewrite it as a small memory, process, descriptor, or thread story.

- translation stages
- main and command-line arguments
- standard streams
- format strings
- checked input
- manual pages

- warnings as feedback
- address-value-dereference reasoning

1.10.2 Expanded API cards

Expanded API card: `printf`

Purpose Print formatted output to stdout.

Header `stdio.h`

Simplified prototype
`int printf(const char *format, ...);`

Parameter meanings
 The format string controls how later arguments are interpreted; each conversion requires a matching promoted type.

Return value Number of characters printed, or negative on output error.

Failure behaviour
 Bad stream state can make output fail; mismatched arguments create undefined behaviour.

Trap The compiler cannot always save a bad format string, especially when it is not a literal.

```
1 printf("count=%d\n", count);
2 printf("ptr=%p\n", (void *)ptr);
```

Listing 8: Small example for `printf`

Expanded API card: `scanf`

Purpose Read formatted input from stdin into caller-provided storage.

Header `stdio.h`

Simplified prototype
`int scanf(const char *format, ...);`

Parameter meanings
 Most conversion arguments are addresses because `scanf` writes into variables supplied by the caller.

Return value Number of successful assignments, or EOF before the first assignment.

Failure behaviour
 Failed conversions leave destination variables unchanged.

Trap A string conversion without a width can overflow the destination buffer.

```
1 int n;
2 if (scanf("%d", &n) != 1) {
3     fprintf(stderr, "bad input\n");
4 }
```

Listing 9: Small example for `scanf`

Expanded API card: getchar

Purpose Read one character from stdin.

Header `stdio.h`

Simplified prototype
`int getchar(void);`

Parameter meanings
No arguments. The result must be stored as `int`, not `char`, because EOF is outside the unsigned `char` range.

Return value Character value converted to `int`, or EOF.

Failure behaviour
EOF can mean end of file or error; `ferror` can distinguish when needed.

Trap Using `char` loses the ability to represent EOF reliably.

```
1 int c;  
2 while ((c = getchar()) != EOF) {  
3     putchar(c);  
4 }
```

Listing 10: Small example for `getchar`

1.10.3 Tutorial extractions with full reasoning

1.10.4 Pointer print trace

What the question is testing The task checks whether a student can separate the value stored in an `int`, the address of that `int`, and the value stored inside a pointer object.

Required background You need the address operator, dereference operator, and the rule that pointer output uses `%p` with a void pointer cast.

```
1 int x = 10;  
2 int *p = &x;  
3 int **pp = &p;  
4 printf("%d\n", x);  
5 printf("%p\n", (void *)p);  
6 printf("%d\n", **pp);
```

Listing 11: Tutorial extraction: Pointer print trace

Step-by-step reasoning

1. `x` names the integer object and initially stores 10.
2. `p` is a different object whose value is the address of `x`.
3. `pp` stores the address of `p`, so two dereferences reach `x`.
4. Printing `p` and printing the address of `x` should describe the same location.

Common mistakes

- Using %d for pointer values.
- Thinking p and x are the same object.
- Forgetting that pp needs two dereferences to reach the int.

Takeaway If you can draw boxes for x, p, and pp, the later linked-list diagrams use the same skill.

1.10.5 Compile-run-debug loop

What the question is testing The task checks whether the student treats gcc diagnostics as part of the development process.

Required background You need the shell command shape, the purpose of -Wall -Wextra -Werror, and the difference between compile and link failures.

```
1 /* Build */
2 /* gcc -Wall -Wextra -Werror -g -O0 main.c -o main */
3 /* Run */
4 /* ./main */
```

Listing 12: Tutorial extraction: Compile-run-debug loop

Step-by-step reasoning

1. Compile with warnings enabled.
2. If the compiler rejects syntax or types, fix the first diagnostic first.
3. If undefined references appear, move to linking reasoning and check object files or libraries.
4. Only after the executable runs should you reason about runtime state.

Common mistakes

- Compiling without -o and then running the wrong old executable.
- Ignoring the first warning because later lines look more dramatic.
- Treating a linker error as a C syntax error.

Takeaway A clean workflow classifies failure by stage before changing code.

1.10.6 Exam-readiness prompts

- Before answering, identify the object or resource whose lifetime is being tested.
- State the API success and failure return values before using the result.
- Draw the relevant storage, descriptor table, process tree, or thread-shared invariant.
- Name the common trap in the question and show how the code avoids it.
- Finish with the cleanup or synchronisation step that makes the example complete.

1.10.7 Depth takeaway

This expansion is intentionally slower than a checklist. The final exam rewards the student who can explain why a line is correct, what would break if one return value or close call were missing, and how the same pattern reappears in a later week.

2 Week 2: Memory & Pointers

2.1 Core idea

Memory is a sequence of bytes. A variable reserves bytes and gives them a type and a name. A pointer is a value whose interpretation is an address. Arrays and pointers are closely related because array expressions often decay to pointers, but an array object and a pointer variable are not the same thing.

2.2 Representation and types

The same byte pattern can represent different things depending on type. A byte can be an unsigned number, a signed number, a character, a permission bitmask, or part of an address. C does not store rich runtime type information for ordinary objects; the compiler uses static type information to generate operations.

```
1 unsigned char u = 255;
2 signed char s = -1;      /* often the same bits as 255 in one byte */
3 char ch = '6';          /* character encoding */
```

Listing 13: Same bits, different interpretation

2.3 Pointers

A pointer declaration names the pointed-to type.

```
1 int value = 42;
2 int *p = &value;
3
4 *p = 99;          /* modifies value */
5 printf("%d\n", value); /* prints 99 */
```

Listing 14: Pointer basics

The type `int *` means pointer to `int`. Pointer arithmetic is scaled by the pointed-to type. If `p` is an `int *`, then `p + 1` moves by `sizeof(int)` bytes.

NULL

- **Context/header:** `<stddef.h>`, `<stdio.h>`, `<stdlib.h>` and others define it.
- **Form:** `int *p = NULL;`
- **Purpose:** A null pointer constant used to mean “points to no object”.
- **Result/trap:** Dereferencing `NULL` is invalid. Use it as a sentinel and after `free`, but do not confuse it with the character `'\0'`.

sizeof

- **Context/header:** Built-in compile-time operator for object/type size.
- **Form:** `sizeof object`; `sizeof(type)`
- **Purpose:** Returns the size in bytes as a `size_t`. It is essential for allocation and layout reasoning.
- **Result/trap:** On an array object, `sizeof array` gives the whole array size. On a pointer, it gives the pointer variable size, not the allocation or pointed-to array length.

2.4 Arrays versus pointers

```
1 int a[5] = {10, 20, 30, 40, 50};
2 int *p = a;
3
4 printf("%zu\n", sizeof a);      /* 5 * sizeof(int) */
5 printf("%zu\n", sizeof p);      /* size of pointer */
6 printf("%d\n", *(p + 2));       /* 30 */
7 printf("%d\n", a[2]);           /* also 30 */
```

Listing 15: Array object versus pointer variable

The expression `a[i]` is defined in terms of pointer arithmetic: it means `*(a + i)`. But the array object `a` is not a modifiable pointer variable; you cannot assign `a = p`.

2.5 Pointer safety rules

- Initialise pointers before using them.
- Only dereference a pointer when it points to a live object.
- Do not return pointers to local stack variables.
- Keep array bounds separate from the pointer value.
- Prefer `sizeof *ptr` in allocation so the expression follows the pointer type.

Common pitfalls

- Confusing `p` and `*p`: address versus object.
- Assuming `sizeof(ptr)` knows how many elements were allocated.
- Dereferencing an uninitialised pointer.
- Moving a pointer outside its array and then dereferencing it.
- Comparing pointer values from unrelated objects as if they were normal numbers.

2.6 Tutorial Extractions

2.6.1 Arrays, pointer decay, and sizeof

This tutorial pattern tests the difference between an array object in scope and a pointer parameter.

```
1 #include <stdio.h>
2
3 void inspect(int xs[]) {
4     printf("inside function: %zu\n", sizeof xs);
5 }
```

```

5 }
6
7 int main(void) {
8     int xs[10];
9     printf("in main: %zu\n", sizeof xs);
10    inspect(xs);
11 }

```

Listing 16: sizeof changes after array-to-pointer decay

Inside `main`, `xs` is an array object, so `sizeof xs` is space for ten integers. In a function parameter, `int xs[]` is adjusted to `int *xs`; `sizeof xs` is only pointer size. The reusable pattern is: pass the length separately.

```

1 void fill_zero(int *xs, size_t n) {
2     for (size_t i = 0; i < n; i++) {
3         xs[i] = 0;
4     }
5 }

```

Listing 17: Length travels separately

2.6.2 Safer input with fgets

fgets

- **Context/header:** `#include <stdio.h>`.
- **Form:** `char *fgets(char *s, int size, FILE *stream);`
- **Purpose:** Reads at most `size - 1` characters, stores a terminating null byte, and stops at newline or EOF.
- **Result/trap:** Returns `s` on success or `NULL` on EOF/error before any character is read. The newline may remain in the buffer.

```

1 char line[128];
2
3 if (fgets(line, sizeof line, stdin) == NULL) {
4     fprintf(stderr, "no input\n");
5 } else {
6     printf("line: %s", line);
7 }

```

Listing 18: Bounded line input

This is safer than an unbounded `scanf("%s", line)` because the destination size is part of the call.

Week takeaway

- A pointer stores an address; dereferencing follows that address.
- Arrays decay to pointers in many expressions, but `sizeof` reveals the difference.
- Pass lengths explicitly.
- Treat `NULL`, `sizeof`, and pointer arithmetic as exam basics.

2.7 Full-depth expansion pass

Week 2 must stay slow and concrete. Memory is bytes, types interpret bytes, arrays are contiguous objects, and pointers are values that designate objects only while lifetime and bounds remain valid.

2.7.1 Teaching-density checklist

Read this week using the original notes standard: each item below should be explainable in words, recognisable in code, and connectable to a failure mode. If an item feels like a vocabulary word only, rewrite it as a small memory, process, descriptor, or thread story.

- bytes and object representation
- pointer types
- NULL
- array-to-pointer decay
- sizeof on arrays versus pointers
- pointer arithmetic
- lifetime
- aliasing
- command-line numeric parsing

2.7.2 Expanded API cards

Expanded API card: sizeof

Purpose Obtain the size in bytes of an object expression or type.

Header C operator

Simplified prototype
sizeof expression; sizeof(type)

Parameter meanings
The operand is either an expression or a parenthesised type name. Most expressions are not evaluated.

Return value A size_t byte count.

Failure behaviour
It does not fail at runtime.

Trap On a pointer, it reports pointer-object size, not the length of the allocation or array.

```
1 int xs[10];
2 printf("%zu\n", sizeof xs);
3 printf("%zu\n", sizeof xs[0]);
```

Listing 19: Small example for sizeof

Expanded API card: `fgets`

Purpose	Read a bounded line from a stream into a char buffer.
Header	<code>stdio.h</code>
Simplified prototype	<code>char *fgets(char *s, int size, FILE *stream);</code>
Parameter meanings	<code>s</code> is writable storage, <code>size</code> is total buffer capacity, <code>stream</code> is the source.
Return value	<code>s</code> on success, <code>NULL</code> on EOF or error before a character is read.
Failure behaviour	Long lines are split across calls; the newline may remain in the buffer.
Trap	<code>size</code> includes the terminator, so at most <code>size - 1</code> ordinary characters are stored.

```
1 char line[128];
2 if (fgets(line, sizeof line, stdin) != NULL) {
3     fputs(line, stdout);
4 }
```

Listing 20: Small example for `fgets`

Expanded API card: `strtol`

Purpose	Convert a string to a long while exposing where parsing stopped.
Header	<code>stdlib.h</code>
Simplified prototype	<code>long strtol(const char *nptr, char **endptr, int base);</code>
Parameter meanings	<code>nptr</code> is input, <code>endptr</code> receives the first unconverted character, <code>base</code> controls radix.
Return value	Converted long value.
Failure behaviour	Overflow sets <code>errno</code> to <code>ERANGE</code> ; no digits leave <code>endptr</code> equal to <code>nptr</code> .
Trap	<code>atoi</code> hides too many errors for serious command-line parsing.

```
1 char *end;
2 errno = 0;
3 long n = strtol(argv[1], &end, 10);
4 if (errno || end == argv[1] || *end != 0) return 1;
```

Listing 21: Small example for `strtol`

2.7.3 Tutorial extractions with full reasoning

2.7.4 Array decay and length passing

What the question is testing The task checks why a callee cannot discover an array length from a pointer parameter.

Required background You need sizeof rules, function parameter adjustment, and explicit size arguments.

```
1 void clear(int *xs, size_t n) {  
2     for (size_t i = 0; i < n; i++) {  
3         xs[i] = 0;  
4     }  
5 }
```

Listing 22: Tutorial extraction: Array decay and length passing

Step-by-step reasoning

1. In the caller, sizeof on a real array sees the whole object.
2. When passed to clear, the array expression decays to a pointer.
3. The function parameter int *xs stores only an address.
4. The element count must be passed separately.

Common mistakes

- Using sizeof xs inside clear.
- Using int for sizes and comparing with size_t.
- Assuming C arrays carry runtime length metadata.

Takeaway A pointer tells where elements begin, not how many elements exist.

2.7.5 Pointer alias update

What the question is testing The task checks whether copying a pointer creates a second path to the same object, not a second object.

Required background You need pointer assignment and dereference assignment.

```
1 int a = 3;  
2 int b = 7;  
3 int *p = &a;  
4 int *q = p;  
5 *q = 11;  
6 p = &b;  
7 *p = *q + 1;
```

Listing 23: Tutorial extraction: Pointer alias update

Step-by-step reasoning

1. p first stores the address of a.
2. q receives a copy of that same address.
3. Writing through q changes a to 11.
4. Changing p to point at b does not move q.
5. The final assignment stores 12 in b.

Common mistakes

- Thinking q tracks p after `q = p`.
- Writing `p = b` instead of `p = &b`.
- Forgetting which object each pointer currently designates.

Takeaway Pointer assignment copies an address value at a moment in time.

2.7.6 Exam-readiness prompts

- Before answering, identify the object or resource whose lifetime is being tested.
- State the API success and failure return values before using the result.
- Draw the relevant storage, descriptor table, process tree, or thread-shared invariant.
- Name the common trap in the question and show how the code avoids it.
- Finish with the cleanup or synchronisation step that makes the example complete.

2.7.7 Depth takeaway

This expansion is intentionally slower than a checklist. The final exam rewards the student who can explain why a line is correct, what would break if one return value or close call were missing, and how the same pattern reappears in a later week.

3 Week 3: String Manipulation

3.1 Core idea

A C string is not a special object. It is a sequence of `char` values terminated by the first null character `'\0'`. All standard string functions rely on that invariant. If a character array is not null-terminated, string functions can read beyond the intended object.

3.2 String storage

```
1 char a[] = "hello";      /* {'h','e','l','l','o','\0'} */
2 char *p = "hello";      /* pointer to string literal */
```

Listing 24: String literal and array string

`a` is a mutable array initialised from the literal. `p` points at a string literal; modifying the literal through `p` is undefined behaviour.

3.3 Length and bounded length

strlen

- **Context/header:** `##include <string.h>`.
- **Form:** `size_t strlen(const char *s);`
- **Purpose:** Counts characters before the first `'\\0'`.
- **Result/trap:** It does not know the allocated buffer size. If there is no terminator, it keeps reading into unrelated memory.

strnlen

- **Context/header:** `##include <string.h>` on POSIX systems.
- **Form:** `size_t strnlen(const char *s, size_t maxlen);`
- **Purpose:** Counts up to `maxlen` characters or the first terminator.
- **Result/trap:** Useful when input may not be properly terminated. It still assumes `s` points to at least `maxlen` readable bytes.

```
1 printf("%zu\n", strlen(""));           /* 0 */
2 printf("%zu\n", strlen("abc\n"));      /* 4 */
3 printf("%zu\n", strlen("lorem\0ipsum")); /* 5 */
```

Listing 25: `strlen` stops at the first null byte

3.4 Copying strings

strcpy

- **Context/header:** `##include <string.h>`.
- **Form:** `char *strcpy(char *dest, const char *src);`
- **Purpose:** Copies `src`, including its terminating null byte, into `dest`.
- **Result/trap:** No bound is checked. The caller must guarantee destination capacity. This is a common buffer overflow source.

strncpy

- **Context/header:** `##include <string.h>`.
- **Form:** `char *strncpy(char *dest, const char *src, size_t n);`
- **Purpose:** Copies at most `n` characters. If `src` is shorter, pads with null bytes.
- **Result/trap:** If `src` length is at least `n`, the destination is not null-terminated. This surprises many students.

```
1 char dest[16];
2
3 strncpy(dest, src, sizeof dest - 1);
4 dest[sizeof dest - 1] = '\\0';
```

Listing 26: Using `strncpy` requires explicit termination

3.5 Comparing strings

strcmp

- **Context/header:** `##include <string.h>`.
- **Form:** `int strcmp(const char *s1, const char *s2);`
- **Purpose:** Lexicographically compares two null-terminated strings.
- **Result/trap:** Returns 0 when equal, negative if the first differing character in `s1` is smaller, positive if larger. Do not compare string contents with `==`.

`strncmp`

- **Context/header:** `##include <string.h>`.
- **Form:** `int strncmp(const char *s1, const char *s2, size_t n);`
- **Purpose:** Compares at most `n` characters.
- **Result/trap:** A return value of 0 means equal for the checked prefix, not necessarily equal full strings.

```

1 char a[] = "tofu";
2 char b[] = "tofu";
3
4 if (a == b) {
5     /* wrong: compares addresses */
6 }
7
8 if (strcmp(a, b) == 0) {
9     /* right: compares contents */
10 }

```

Listing 27: String comparison

3.6 Character stream transformation

`toupper`

- **Context/header:** `##include <ctype.h>`.
- **Form:** `int toupper(int c);`
- **Purpose:** Converts lowercase letters to uppercase and leaves other characters unchanged.
- **Result/trap:** Pass either EOF or a value representable as `unsigned char`. Store `getchar` results in `int`.

```

1 #include <ctype.h>
2 #include <stdio.h>
3
4 int main(void) {
5     int c;
6     while ((c = getchar()) != EOF) {
7         putchar(toupper(c));
8     }
9     return 0;
10 }

```

Listing 28: Uppercase filter

3.7 Bitwise operations and math tooling

Week 3 also introduces bitwise operators and `math.h`. Bitwise operators act on the representation itself: `&`, `|`, `^`, `~`, `«`, and `»`. Programs using math library functions commonly need `-lm` at link time.

```
1 gcc area.c -o area -lm
```

Listing 29: Linking the maths library

Common pitfalls

- Treating a character array as a string without a `'\\0'` terminator.
- Assuming `strncpy` always terminates.
- Comparing strings with `==`.
- Forgetting that `strlen` stops at the first null byte, not at array capacity.
- Passing a `char` directly to `toupper` when it may be negative.

3.8 Tutorial Extractions

3.8.1 Implementing strlen

```
1 #include <stddef.h>
2
3 size_t my_strlen(const char *s) {
4     size_t len = 0;
5     while (s[len] != '\\0') {
6         len++;
7     }
8     return len;
9 }
```

Listing 30: Minimal strlen implementation

The concept tested is the string model: the array may be larger than the visible string, but the first null byte ends the string. The function uses `const char *` because it only reads the data.

3.8.2 Bounded string comparison

```
1 int string_compare(const char *a, const char *b, size_t limit) {
2     size_t i = 0;
3
4     while (i < limit && a[i] != '\\0' && b[i] != '\\0') {
5         if (a[i] != b[i]) {
6             return (unsigned char)a[i] - (unsigned char)b[i];
7         }
8         i++;
9     }
10
11     if (i == limit) {
12         return 0;
13     }
14     return (unsigned char)a[i] - (unsigned char)b[i];
}
```

Listing 31: Compare up to a bound

The reusable idea is to combine three stopping conditions: reached the bound, reached the end of the first string, or reached the end of the second string.

Week takeaway

- A C string is a null-terminated character sequence.
- String APIs trust the caller to provide valid terminated memory.
- Bounded functions reduce risk but do not remove the need to reason.
- Per-character filters are cleanly written with `getchar`, `toupper`, and `putchar`.

3.9 Full-depth expansion pass

String manipulation should be as detailed as memory manipulation because a C string is only a memory convention. The first zero byte ends the string, regardless of the physical array size.

3.9.1 Teaching-density checklist

Read this week using the original notes standard: each item below should be explainable in words, recognisable in code, and connectable to a failure mode. If an item feels like a vocabulary word only, rewrite it as a small memory, process, descriptor, or thread story.

- string literals
- char arrays
- null terminator
- buffer capacity
- strlen scanning
- copying and termination
- lexicographic comparison
- ctype safety
- tokenisation side effects
- bitwise operators

3.9.2 Expanded API cards**Expanded API card: strlen**

Purpose Count bytes before the first string terminator.

Header `string.h`

Simplified prototype
`size_t strlen(const char *s);`

Parameter meanings
 s must point at a valid null-terminated string.

Return value Length excluding the terminator.

Failure behaviour

Missing terminator can make it read beyond the intended object.

Trap

It reports logical string length, not array capacity.

```
1 size_t n = strlen(name);  
2 printf("%zu\n", n);
```

Listing 32: Small example for strlen

Expanded API card: strncpy

Purpose

Copy up to n characters, padding with zero bytes if src is short.

Header

string.h

Simplified prototype

char *strncpy(char *dest, const char *src, size_t n);

Parameter meanings

dest is destination storage, src is input string, n is a byte count.

Return value

dest.

Failure behaviour

If src length is at least n, no terminator is appended.

Trap

It is not automatically safe strcpy; explicit termination is often still required.

```
1 char dest[16];  
2 strncpy(dest, src, sizeof dest - 1);  
3 dest[sizeof dest - 1] = 0;
```

Listing 33: Small example for strncpy

Expanded API card: strcmp

Purpose

Compare string contents lexicographically.

Header

string.h

Simplified prototype

int strcmp(const char *s1, const char *s2);

Parameter meanings

Both arguments must be valid terminated strings.

Return value

Zero if equal, negative or positive according to first differing character.

Failure behaviour

Invalid strings produce undefined behaviour.

Trap

== compares addresses, not contents.

```

1 if (strcmp(cmd, "quit") == 0) {
2     done = 1;
3 }

```

Listing 34: Small example for strcmp

3.9.3 Tutorial extractions with full reasoning

3.9.4 Implement strlen carefully

What the question is testing The task checks that a string ends at the first terminator and that the terminator is not counted.

Required background You need a `size_t` index and read-only traversal through `const char *`.

```

1 size_t my_strlen(const char *s) {
2     size_t n = 0;
3     while (s[n] != 0) {
4         n++;
5     }
6     return n;
7 }

```

Listing 35: Tutorial extraction: Implement strlen carefully

Step-by-step reasoning

1. Start at zero so the empty string works.
2. Read `s[n]` before incrementing.
3. Stop exactly at the terminator.
4. Return `size_t` because lengths are sizes.

Common mistakes

- Counting the terminator.
- Writing through `s`.
- Assuming the physical array length is available.

Takeaway Most string code is a bounded or unbounded scan with a precise stopping condition.

3.9.5 Run-length encoding boundary

What the question is testing The task checks grouped scanning and the final flush after the loop.

Required background You need a current character, a count, and handling for an empty string.

```

1 void encode(const char *s) {
2     if (s[0] == 0) return;
3     char cur = s[0];
4     int count = 1;
5     for (size_t i = 1; s[i] != 0; i++) {
6         if (s[i] == cur) count++;
7         else { printf("%c%d", cur, count); cur = s[i]; count = 1; }
8     }
9     printf("%c%d", cur, count);
10 }

```

Listing 36: Tutorial extraction: Run-length encoding boundary

Step-by-step reasoning

1. Handle empty input before starting the first run.
2. Initialise the first run from `s[0]`.
3. When a different character arrives, print the old run.
4. After the loop, print the last run because no later character will trigger it.

Common mistakes

- Forgetting the last run.
- Double-counting the first character.
- Failing on empty input.

Takeaway End of input is also a boundary that often requires explicit work.

3.9.6 Exam-readiness prompts

- Before answering, identify the object or resource whose lifetime is being tested.
- State the API success and failure return values before using the result.
- Draw the relevant storage, descriptor table, process tree, or thread-shared invariant.
- Name the common trap in the question and show how the code avoids it.
- Finish with the cleanup or synchronisation step that makes the example complete.

3.9.7 Depth takeaway

This expansion is intentionally slower than a checklist. The final exam rewards the student who can explain why a line is correct, what would break if one return value or close call were missing, and how the same pattern reappears in a later week.

4 Week 4: Enums, Structs, Unions, Files

4.1 Core idea

This week adds richer data organisation and file streams. `enum` gives readable named integer values. `struct` groups fields into one object. `union` overlays several views on the same memory. Files introduce persistent data and stream buffering through `FILE *`.

4.2 Enums

enum

- **Context/header:** C language type constructor.
- **Form:** `enum day_name \{ SUN, MON, TUE \};`
- **Purpose:** Creates named integer constants for a fixed set of choices.
- **Result/trap:** Enums improve readability but are still close to integers in C. Do not assume runtime enforcement of only legal values.

```
1 enum month_name {  
2     JAN, FEB, MAR, APR, MAY, JUN,  
3     JUL, AUG, SEP, OCT, NOV, DEC,  
4     MONTH_UNDEF  
5 };
```

Listing 37: Enums for readable states

4.3 Structs and typedef

struct

- **Context/header:** C language aggregate type.
- **Form:** `struct date \{ int day; int month; int year; \};`
- **Purpose:** Groups related fields, possibly of different types, into one object.
- **Result/trap:** Passing a struct by value copies the whole struct. Use pointers for large objects or when mutation is needed.

typedef

- **Context/header:** C language declaration feature.
- **Form:** `typedef struct date Date;`
- **Purpose:** Creates an alias for a type. Commonly used to avoid repeatedly writing `struct` name.
- **Result/trap:** A typedef is not a new runtime type; it is a naming convenience. Overusing typedef can hide pointer-ness or ownership.

```
1 typedef struct date {  
2     enum month_name month;  
3     int day;  
4     int year;  
5 } Date;  
6  
7 Date deadline = {MAY, 8, 2026};  
8 Date *p = &deadline;  
9  
10 printf("%d\n", deadline.year); /* object: dot */  
11 printf("%d\n", p->year);      /* pointer: arrow */
```

Listing 38: Struct object versus pointer

4.4 Standard library structs

struct tm

- **Context/header:** `##include <time.h>`.
- **Form:** `struct tm *localtime(const time_t *timep);`
- **Purpose:** Represents broken-down calendar time with fields such as `tm_mday`, `tm_mon`, and `tm_year`.
- **Result/trap:** `localtime` may return a pointer to statically allocated storage that can be overwritten by later time calls. Months are commonly zero-based and years are since 1900.

struct stat

- **Context/header:** `##include <sys/stat.h>`, system interfaces.
- **Form:** `int stat(const char *path, struct stat *buf);`
- **Purpose:** Stores file metadata such as mode, size, and timestamps when using filesystem queries.
- **Result/trap:** Check the return value. On failure, `-1` is returned and `errno` is set. This appears as supporting systems context around file metadata.

4.5 Padding and alignment

Struct size is not always the sum of field sizes. The compiler may insert padding so fields are placed at addresses suitable for the hardware.

```
1 struct compact_maybe {
2     int x;
3     short s;
4     char c;
5 };
6
7 struct padded_maybe {
8     char c;
9     int x;
10    short s;
11 };
```

Listing 39: Field order can affect padding

The exam reasoning pattern is: `sizeof(struct)` includes padding. Binary file output of structs may therefore include padding bytes, and padding rules are compiler/platform dependent.

4.6 Unions and tagged unions

union

- **Context/header:** C language aggregate type.
- **Form:** `union value { int i; double d; };`
- **Purpose:** Stores several possible members in the same memory region. The size is enough for the largest member plus alignment.
- **Result/trap:** The union does not remember which member is active. Reading a different member than the one most recently written is low-level representation work and can be non-portable.

```

1 enum holding_type { BOOK, FILM };
2
3 struct catalog {
4     char *title;
5     enum holding_type type;
6     union {
7         struct {
8             char *author;
9             char *isbn;
10        } book;
11        struct {
12            char *director;
13            char *producer;
14        } film;
15    } info;
16 };
17
18 void print_holding(const struct catalog *item) {
19     switch (item->type) {
20     case BOOK:
21         printf("book by %s\n", item->info.book.author);
22         break;
23     case FILM:
24         printf("film by %s\n", item->info.film.director);
25         break;
26     }
27 }

```

Listing 40: Tagged union pattern

The enum tag tells the program which union member is currently meaningful.

4.7 Bitfields and bit operations

bitfields

- **Context/header:** C struct field syntax.
- **Form:** unsigned mode : 3;
- **Purpose:** Stores a field using a specified number of bits, useful for compact flags or device register style data.
- **Result/trap:** Layout, ordering, and padding are implementation-dependent. For portable file formats, masks and shifts are often clearer.

```

1 unsigned short x = 0x9234;
2 unsigned rw    = x >> 15;
3 unsigned dirn  = (x >> 7) & 0xff;
4 unsigned mode  = (x >> 4) & 0x7;

```

Listing 41: Manual bit extraction

4.8 Streams and FILE *

FILE *

- **Context/header:** `##include <stdio.h>`.

- **Form:** `FILE *fp;`
- **Purpose:** An opaque standard-library stream object used by `fopen`, `fgets`, `fprintf`, `fscanf`, and `fclose`.
- **Result/trap:** It is not the same thing as an integer file descriptor. The library may buffer data before it reaches the OS.

`fopen`

- **Context/header:** `##include <stdio.h>.`
- **Form:** `FILE *fopen(const char *path, const char *mode);`
- **Purpose:** Open a file as a stream. Modes include `"r"`, `"w"`, `"a"`, `"rb"`, `"wb"`, and update modes such as `"r+"`.
- **Result/trap:** Returns `NULL` on failure. Always check before using the stream. `"w"` truncates existing files.

`fclose`

- **Context/header:** `##include <stdio.h>.`
- **Form:** `int fclose(FILE *stream);`
- **Purpose:** Flushes buffered output and closes the stream.
- **Result/trap:** Returns 0 on success or EOF on error. After `fclose`, the `FILE *` must not be used.

`fscanf`

- **Context/header:** `##include <stdio.h>.`
- **Form:** `int fscanf(FILE *stream, const char *format, ...);`
- **Purpose:** Formatted input from a chosen stream.
- **Result/trap:** Returns assignments completed. Check it; otherwise old variable values may be used after failed input.

`fflush`

- **Context/header:** `##include <stdio.h>.`
- **Form:** `int fflush(FILE *stream);`
- **Purpose:** For output streams, pushes buffered data toward the underlying file/device.
- **Result/trap:** Use before expecting external observers to see output. Calling it on input streams is not portable ISO C behaviour.

`feof`, `ferror`

- **Context/header:** `##include <stdio.h>.`
- **Form:** `int feof(FILE *stream); int ferror(FILE *stream);`
- **Purpose:** Inspect end-of-file and error indicators on a stream.
- **Result/trap:** Do not use `while (!feof(fp))` as the primary read loop. Try the read, then inspect the result.

```

1 FILE *fp = fopen("numbers.txt", "r");
2 if (fp == NULL) {
3     perror("numbers.txt");

```

```

4     return 1;
5 }
6
7 int x;
8 while (fscanf(fp, "%d", &x) == 1) {
9     printf("%d\n", x);
10 }
11
12 if (ferror(fp)) {
13     fprintf(stderr, "read error\n");
14 }
15 fclose(fp);

```

Listing 42: Correct file-reading loop shape

4.9 Binary files and struct layout

```

1 typedef struct pokemon {
2     char name[100];
3     unsigned level;
4 } Pokemon;
5
6 Pokemon team[] = {
7     {"Bulbasaur", 12},
8     {"Squirtle", 16}
9 };
10
11 FILE *out = fopen("pokemon.dat", "wb");
12 if (out == NULL) {
13     perror("pokemon.dat");
14     return 1;
15 }
16
17 fwrite(team, sizeof team[0], 2, out);
18 fclose(out);

```

Listing 43: Writing raw struct bytes

The reusable warning is that binary output of structs writes the bytes in memory, including padding and machine-specific representation. That is different from text output with `fprintf`.

Common pitfalls

- Using `p.field` when `p` is a pointer; use `p->field`.
- Forgetting that struct assignment and function arguments copy the whole struct.
- Assuming struct binary layout is portable.
- Reading a union without tracking the active member.
- Writing `while (!feof(fp))` instead of checking the read operation.

4.10 Tutorial Extractions

4.10.1 Tagged union as a reusable design pattern

The tutorial-style problem is not just syntax; it asks how to represent one of several possible shapes safely.

```
1 enum value_kind { VALUE_INT, VALUE_STRING };
2
3 struct value {
4     enum value_kind kind;
5     union {
6         int i;
7         char *s;
8     } as;
9 };
10
11 void print_value(const struct value *v) {
12     if (v->kind == VALUE_INT) {
13         printf("%d\n", v->as.i);
14     } else {
15         printf("%s\n", v->as.s);
16     }
17 }
```

Listing 44: Tagged value pattern

The tag and the union must be updated together. If the tag says string but the integer member was last written, the program's logic has lost the invariant.

4.10.2 Text file read pattern

```
1 char name[64];
2 int mark;
3
4 while (fscanf(fp, "%63s %d", name, &mark) == 2) {
5     printf("%s: %d\n", name, mark);
6 }
```

Listing 45: Read records from a text file

The width `%63s` leaves space for the null terminator in `name[64]`. The return value must be 2 because the loop needs both fields.

Week takeaway

- `enum`, `struct`, `union`, and `typedef` are data modelling tools.
- Struct layout includes padding; binary I/O of structs is representation-dependent.
- Tagged unions are the safe pattern for variant data.
- File stream code is `fopen`, checked read/write calls, error checks, then `fclose`.

4.11 Full-depth expansion pass

Week 4 keeps the original note style by treating data modelling as memory modelling. Structs, unions, padding, and files are not just syntax; they define how bytes are grouped, interpreted, copied, and persisted.

4.11.1 Teaching-density checklist

Read this week using the original notes standard: each item below should be explainable in words, recognisable in code, and connectable to a failure mode. If an item feels like a vocabulary word only, rewrite it as a small memory, process, descriptor, or thread story.

- enum as named integers
- struct field access
- typedef tradeoffs
- padding and alignment
- union active member
- tagged union invariant
- FILE streams
- text versus binary files
- fscanf return counts
- raw struct portability

4.11.2 Expanded API cards

Expanded API card: fopen

Purpose Open a path as a buffered standard I/O stream.

Header `stdio.h`

Simplified prototype
`FILE *fopen(const char *path, const char *mode);`

Parameter meanings
path names the file, mode selects reading, writing, append, binary, or update behaviour.

Return value FILE pointer on success, NULL on failure.

Failure behaviour
Missing files, permissions, and invalid modes can fail.

Trap Mode `w` truncates an existing file.

```
1 FILE *fp = fopen(path, "r");  
2 if (fp == NULL) { perror(path); }
```

Listing 46: Small example for fopen

Expanded API card: fscanf

Purpose Read formatted data from a chosen stream.

Header stdio.h

Simplified prototype

```
int fscanf(FILE *stream, const char *format, ...);
```

Parameter meanings

Destination arguments are usually addresses; string conversions should include a width.

Return value Number of assignments completed, or EOF.

Failure behaviour

Partial records are common when input is malformed.

Trap Loop on the exact expected assignment count.

```
1 char name[64];
2 int mark;
3 while (fscanf(fp, "%63s %d", name, &mark) == 2) {
4     printf("%s %d\n", name, mark);
5 }
```

Listing 47: Small example for fscanf

Expanded API card: fread

Purpose Read binary items into memory.

Header stdio.h

Simplified prototype

```
size_t fread(void *ptr, size_t size, size_t nmemb, FILE *stream);
```

Parameter meanings

ptr is destination, size is bytes per item, nmemb is item count.

Return value Number of complete items read.

Failure behaviour

Short reads require feof or ferror to distinguish EOF and error.

Trap Raw structs include padding and machine representation.

```
1 struct rec r;
2 if (fread(&r, sizeof r, 1, fp) == 1) {
3     use(&r);
4 }
```

Listing 48: Small example for fread

4.11.3 Tutorial extractions with full reasoning

4.11.4 Tagged union invariant

What the question is testing The task checks that a union needs an external tag to be safely interpreted.

Required background You need enum states, a union payload, and a switch or if on the tag.

```
1 enum kind { VALUE_INT, VALUE_WORD };
2 struct value {
3     enum kind kind;
4     union { int i; char word[32]; } as;
5 };
```

Listing 49: Tutorial extraction: Tagged union invariant

Step-by-step reasoning

1. Create an enum for the possible active cases.
2. Store the enum next to the union.
3. Write the tag whenever writing the payload.
4. Read only the member selected by the tag.

Common mistakes

- Assuming the union remembers the active member.
- Using a union when all fields are needed simultaneously.
- Updating payload and tag separately in inconsistent paths.

Takeaway A tagged union is a programmer-maintained invariant.

4.11.5 feof loop correction

What the question is testing The task checks that EOF is discovered by attempting a read, not predicted before the read.

Required background You need fscanf return values and optional error after the loop.

```
1 int x;
2 while (fscanf(fp, "%d", &x) == 1) {
3     printf("%d\n", x);
4 }
5 if (ferror(fp)) {
6     perror("read");
7 }
```

Listing 50: Tutorial extraction: feof loop correction

Step-by-step reasoning

1. Attempt the conversion.
2. Enter the body only if the conversion assigned the value.
3. End the loop on EOF or malformed input.
4. Inspect `ferror` if a true I/O error matters.

Common mistakes

- Writing while (`!feof(fp)`).
- Using `x` after a failed conversion.
- Forgetting width limits for strings.

Takeaway The read operation should control the loop.

4.11.6 Exam-readiness prompts

- Before answering, identify the object or resource whose lifetime is being tested.
- State the API success and failure return values before using the result.
- Draw the relevant storage, descriptor table, process tree, or thread-shared invariant.
- Name the common trap in the question and show how the code avoids it.
- Finish with the cleanup or synchronisation step that makes the example complete.

4.11.7 Depth takeaway

This expansion is intentionally slower than a checklist. The final exam rewards the student who can explain why a line is correct, what would break if one return value or close call were missing, and how the same pattern reappears in a later week.

5 Week 5: Dynamic Memory and Linked Lists

5.1 Core idea

Stack memory is automatic and tied to function calls. Heap memory is explicit and lasts until the programmer frees it. Linked lists are a natural heap data structure: each node stores data plus a pointer to the next node.

5.2 Stack, static storage, and heap

- Stack: local variables, parameters, return addresses, temporary call state.
- Static/global storage: globals and `static` variables; usually lifetime is the whole program.
- Heap: memory requested at runtime with allocation functions.
- Code/text: executable instructions.

Never return a pointer to a local stack variable. That storage disappears when the function returns.

5.3 Allocation APIs

`malloc`

- **Context/header:** `##include <stdlib.h>`.
- **Form:** `void *malloc(size_t size);`
- **Purpose:** Allocates `size` bytes of uninitialised heap memory.
- **Result/trap:** Returns `NULL` on failure. The bytes have indeterminate values, so initialise before reading.

`calloc`

- **Context/header:** `##include <stdlib.h>`.
- **Form:** `void *calloc(size_t nmemb, size_t size);`
- **Purpose:** Allocates space for an array of `nmemb` objects and zero-initialises the bytes.
- **Result/trap:** Returns `NULL` on failure. Still check for overflow-sensitive size calculations in serious code.

`realloc`

- **Context/header:** `##include <stdlib.h>`.
- **Form:** `void *realloc(void *ptr, size_t size);`
- **Purpose:** Resizes a heap allocation, possibly moving it. Existing bytes are preserved up to the smaller old/new size.
- **Result/trap:** Do not assign directly to the only pointer unless you are prepared for failure. If `realloc` returns `NULL`, the old allocation is still live.

`free`

- **Context/header:** `##include <stdlib.h>`.
- **Form:** `void free(void *ptr);`
- **Purpose:** Releases heap memory previously returned by allocation functions.
- **Result/trap:** Only free heap allocations once. `free(NULL)` is safe. Using memory after `free` is invalid.

```
1 size_t capacity = 4;
2 size_t len = 0;
3 int *xs = calloc(capacity, sizeof *xs);
4 if (xs == NULL) {
5     perror("calloc");
6     return 1;
7 }
8
9 if (len == capacity) {
10     size_t new_capacity = capacity * 2;
11     int *tmp = realloc(xs, new_capacity * sizeof *xs);
12     if (tmp == NULL) {
13         free(xs);
14         perror("realloc");
15         return 1;
16     }
17     xs = tmp;
18     capacity = new_capacity;
```



```

19 }
20
21 free(xs);

```

Listing 51: Safe realloc pattern

5.4 Memory debugging tools

-fsanitize=address

- **Context/header:** Compiler/runtime instrumentation.
- **Form:** `gcc -g -O0 -fsanitize=address main.c -o main`
- **Purpose:** Builds with AddressSanitizer to detect many invalid memory accesses and leaks at runtime.
- **Result/trap:** Run the program normally after compiling. It changes runtime behaviour slightly, but is very useful for finding use-after-free, out-of-bounds access, and leaks.

gdb

- **Context/header:** Debugger, shell tool.
- **Form:** `gdb ./program`
- **Purpose:** Run, break, step, inspect variables, and view stack frames.
- **Result/trap:** Best with `-g -O0`. Learn `run`, `break`, `next`, `step`, `print`, `backtrace`.

valgrind

- **Context/header:** Dynamic analysis tool.
- **Form:** `valgrind ./program`
- **Purpose:** Runs the program under instrumentation to report invalid reads/writes, leaks, and use of uninitialised values.
- **Result/trap:** Slower than normal execution. Compile with `-g` so reports point to useful lines.

5.5 Linked lists

```

1 struct node {
2     int data;
3     struct node *next;
4 };

```

Listing 52: Singly linked list node

An empty list is NULL. A non-empty list is a chain of nodes ending in NULL. The head pointer is the only way to reach the first node, so code that changes the first node must return the new head or receive a pointer to the head pointer.

```

1 struct node *push_front(struct node *head, int value) {
2     struct node *n = malloc(sizeof *n);
3     if (n == NULL) {
4         perror("malloc");
5         exit(1);
6     }

```

```

7     n->data = value;
8     n->next = head;
9     return n;
10 }

```

Listing 53: Insert at front

```

1 void list_free(struct node *head) {
2     while (head != NULL) {
3         struct node *next = head->next;
4         free(head);
5         head = next;
6     }
7 }

```

Listing 54: Free a list

The order matters: save `head->next` before freeing `head`.

5.6 Double pointer list manipulation

```

1 void list_add(struct node **head, int value) {
2     struct node *n = malloc(sizeof *n);
3     if (n == NULL) {
4         perror("malloc");
5         exit(1);
6     }
7     n->data = value;
8     n->next = *head;
9     *head = n;
10 }

```

Listing 55: Modify caller's head pointer

Use `struct node **head` when the function must change the caller's head pointer itself. This is the same address/value idea from Week 1, now applied to a pointer variable.

Common pitfalls

- Allocating `sizeof(pointer)` instead of `sizeof *pointer`.
- Failing to check allocation results.
- Losing the only pointer to an allocation.
- Freeing a node before saving its `next` pointer.
- Freeing pointer fields that the node does not own.
- Assuming linked lists provide fast indexed access.

5.7 Tutorial Extractions

5.7.1 Dynamic array growth

The dynamic array tutorial pattern tests capacity versus size.

```

1 int append_int(int **arr, size_t *size, size_t *cap, int value) {
2     if (*size == *cap) {
3         size_t new_cap = (*cap == 0) ? 4 : *cap * 2;
4         int *tmp = realloc(*arr, new_cap * sizeof **arr);
5         if (tmp == NULL) {
6             return -1;
7         }
8         *arr = tmp;
9         *cap = new_cap;
10    }
11
12    (*arr)[*size] = value;
13    (*size)++;
14    return 0;
15 }

```

Listing 56: Append with capacity growth

The function takes `int **arr` because `realloc` may return a new address. It takes pointers to `size` and `cap` because it updates the caller's metadata.

5.7.2 Storage duration

```

1 int global_count;
2
3 int sum(const int *x) {
4     static int total = 0;
5     total += *x;
6     return total;
7 }
8
9 int main(void) {
10    int local = 2;
11    int *heap = malloc(sizeof *heap);
12    if (heap == NULL) {
13        return 1;
14    }
15    *heap = 3;
16    sum(&local);
17    sum(&global_count);
18    sum(heap);
19    free(heap);
20 }

```

Listing 57: Storage duration examples

`local` is stack storage. `global_count` and `total` have static storage duration. The pointer variable `heap` is local, but the object it points to lives on the heap until `free`.

Week takeaway

- Heap memory is manual: allocate, check, initialise, use, free.
- `realloc` can move memory, so use a temporary pointer.
- Linked lists are pointer chains; assignment order is part of correctness.
- Double pointers are useful when a function must change the caller's pointer.

5.8 Full-depth expansion pass

The original dynamic-memory chapter should feel practical and exact. Each allocation has a size calculation, an ownership decision, an initialisation step, uses within lifetime, and exactly one release path.

5.8.1 Teaching-density checklist

Read this week using the original notes standard: each item below should be explainable in words, recognisable in code, and connectable to a failure mode. If an item feels like a vocabulary word only, rewrite it as a small memory, process, descriptor, or thread story.

- stack versus heap
- ownership
- malloc uninitialised bytes
- calloc zeroing
- realloc movement
- free and dangling aliases
- linked-list head pointer
- double pointer updates
- leaks
- use-after-free

5.8.2 Expanded API cards

Expanded API card: malloc

Purpose Allocate size bytes of uninitialised heap storage.

Header `stdlib.h`

Simplified prototype
`void *malloc(size_t size);`

Parameter meanings
size is a byte count usually computed with `sizeof`.

Return value Pointer to suitable storage, or `NULL`.

Failure behaviour
Allocation can fail; contents are indeterminate.

Trap `malloc(sizeof pointer)` is a common wrong-size bug.

```
1 struct node *n = malloc(sizeof *n);
2 if (n == NULL) return NULL;
```

Listing 58: Small example for malloc

Expanded API card: realloc

Purpose	Resize an allocation, possibly moving it.
Header	stdlib.h
Simplified prototype	<code>void *realloc(void *ptr, size_t size);</code>
Parameter meanings	ptr is NULL or a previous allocation pointer; size is new byte count.
Return value	New pointer on success, NULL on failure.
Failure behaviour	On failure, the old allocation remains valid.
Trap	Assign to a temporary before overwriting the only pointer.

```
1 int *tmp = realloc(xs, new_cap * sizeof *xs);
2 if (tmp == NULL) { free(xs); return -1; }
3 xs = tmp;
```

Listing 59: Small example for realloc

Expanded API card: free

Purpose	Release heap storage.
Header	stdlib.h
Simplified prototype	<code>void free(void *ptr);</code>
Parameter meanings	ptr is NULL or a currently allocated heap pointer.
Return value	No return value.
Failure behaviour	Double free, freeing stack storage, or using after free are invalid.
Trap	Setting one pointer to NULL does not clear other aliases.

```
1 free(head);
2 head = NULL;
```

Listing 60: Small example for free

5.8.3 Tutorial extractions with full reasoning

5.8.4 Delete from a linked list

What the question is testing The task checks pointer rewiring and ownership order.

Required background You need a pointer to the link being modified and the rule to save next before free.

```
1 int remove_value(struct node **head, int value) {
2     struct node **link = head;
3     while (*link != NULL && (*link)->value != value) {
4         link = &(*link)->next;
5     }
6     if (*link == NULL) return 0;
7     struct node *old = *link;
8     *link = old->next;
9     free(old);
10    return 1;
11 }
```

Listing 61: Tutorial extraction: Delete from a linked list

Step-by-step reasoning

1. link starts as the address of the caller head pointer.
2. Moving link walks to the next field that points at the current node.
3. When found, save the victim node.
4. Rewrite the link to skip the victim.
5. Free the victim after the chain no longer needs it.

Common mistakes

- Freeing before reading next.
- Special-casing head and forgetting to update it.
- Continuing traversal through freed memory.

Takeaway Linked-list code is often clearer when you manipulate links, not just nodes.

5.8.5 Dynamic array append

What the question is testing The task checks length, capacity, and realloc failure safety.

Required background You need metadata pointers because the function may change the caller allocation and counts.

```
1 int append(int **a, size_t *len, size_t *cap, int v) {
2     if (*len == *cap) {
3         size_t next = *cap ? *cap * 2 : 4;
4         int *tmp = realloc(*a, next * sizeof **a);
5         if (tmp == NULL) return -1;
6         *a = tmp;
7         *cap = next;
8     }
9     (*a)[(*len)++] = v;
10    return 0;
11 }
```

Listing 62: Tutorial extraction: Dynamic array append

Step-by-step reasoning

1. Grow before writing when length equals capacity.
2. Use a temporary for realloc.
3. Update caller pointer and capacity only after success.
4. Write the new element and increment length.

Common mistakes

- Losing the old pointer on realloc failure.
- Confusing length with capacity.
- Using `sizeof *a` instead of `sizeof **a`.

Takeaway A dynamic array is an allocation plus trustworthy metadata.

5.8.6 Exam-readiness prompts

- Before answering, identify the object or resource whose lifetime is being tested.
- State the API success and failure return values before using the result.
- Draw the relevant storage, descriptor table, process tree, or thread-shared invariant.
- Name the common trap in the question and show how the code avoids it.
- Finish with the cleanup or synchronisation step that makes the example complete.

5.8.7 Depth takeaway

This expansion is intentionally slower than a checklist. The final exam rewards the student who can explain why a line is correct, what would break if one return value or close call were missing, and how the same pattern reappears in a later week.

6 Week 6: Function Pointers, Signals, errno, Low-level I/O

6.1 Core idea

Compiled code lives at addresses. A function pointer stores the address of executable code and lets a program choose behaviour at runtime. Signals introduce asynchronous control flow. Low-level I/O uses integer file descriptors and system calls rather than `FILE *` streams.

6.2 Function pointers

function pointer syntax

- **Context/header:** C declaration syntax.
- **Form:** `return_type (*name)(parameter_types);`
- **Purpose:** Stores the address of a function with a matching signature.
- **Result/trap:** The parentheses matter. `int *f(void)` is a function returning `int *`; `int (*f)(void)` is a pointer to a function returning `int`.

```

1 int add(int a, int b) {
2     return a + b;
3 }
4
5 int calc(int a, int b, int (*op)(int, int)) {
6     return op(a, b);
7 }
8
9 int result = calc(2, 3, add);

```

Listing 63: Basic function pointer

Function pointers are used for callbacks, error handlers, dispatch tables, and library algorithms.

qsort

- **Context/header:** `#include <stdlib.h>`.
- **Form:** `void qsort(void *base, size_t nmemb, size_t size, int (*compar)(const void *, const void *));`
- **Purpose:** Sorts an array using a caller-provided comparator.
- **Result/trap:** The comparator receives pointers to elements. Cast them to the correct type and return negative, zero, or positive. Avoid subtraction when it can overflow.

```

1 struct fish {
2     char name[32];
3     int weight;
4 };
5
6 int compare_fish_weight(const void *pa, const void *pb) {
7     const struct fish *a = pa;
8     const struct fish *b = pb;
9
10    if (a->weight < b->weight) return -1;
11    if (a->weight > b->weight) return 1;
12    return 0;
13 }
14
15 qsort(fish, nfish, sizeof fish[0], compare_fish_weight);

```

Listing 64: qsort comparator for structs

6.3 Signals

A signal is a software interrupt delivered to a process. Examples include `SIGINT`, `SIGTERM`, `SIGSEGV`, and `SIGKILL`. Some can be caught; `SIGKILL` cannot.

signal

- **Context/header:** `include <signal.h>`.
- **Form:** `void (*signal(int signum, void (*handler)(int)))(int);`
- **Purpose:** Older interface for installing a signal handler.
- **Result/trap:** The lecture treats it as older/deprecated in favour of `sigaction`. It is less explicit and less flexible.

sigaction

- **Context/header:** `##include <signal.h>`.
- **Form:** `int sigaction(int signum, const struct sigaction *act, struct sigaction *oldact);`
- **Purpose:** Preferred interface for installing signal handlers and controlling signal behaviour.
- **Result/trap:** Returns 0 on success, -1 on error and sets `errno`. Initialise the struct, set handler fields deliberately, and remember handlers run asynchronously.

```
1 #include <signal.h>
2 #include <stdio.h>
3 #include <unistd.h>
4
5 static volatile sig_atomic_t interrupted = 0;
6
7 static void handle_int(int signo) {
8     (void)signo;
9     interrupted = 1;
10 }
11
12 int main(void) {
13     struct sigaction act = {0};
14     act.sa_handler = handle_int;
15
16     if (sigaction(SIGINT, &act, NULL) == -1) {
17         perror("sigaction");
18         return 1;
19     }
20
21     while (!interrupted) {
22         pause();
23     }
24     printf("interrupted\n");
25     return 0;
26 }
```

Listing 65: Signal-safe flag pattern

In a handler, set a small flag and return. Do not call arbitrary library functions from a signal handler; many are not async-signal-safe.

6.4 `errno` and diagnostics

`errno`

- **Context/header:** `##include <errno.h>`.
- **Form:** `extern int errno;`
- **Purpose:** Stores detailed error codes for many failed library calls and system calls.
- **Result/trap:** Only inspect `errno` after a function indicates failure. Save it immediately if later calls might overwrite it.

`perror`

- **Context/header:** `##include <stdio.h>`.
- **Form:** `void perror(const char *s);`

- **Purpose:** Prints a message based on the current `errno` to `stderr`.
- **Result/trap:** Call soon after failure. It is for diagnostics, not program logic.

`strerror`

- **Context/header:** `##include <string.h>`.
- **Form:** `char *strerror(int errnum);`
- **Purpose:** Returns a textual description of an error number.
- **Result/trap:** Useful when formatting your own error messages. The returned storage may be static.

```

1 #include <errno.h>
2 #include <stdio.h>
3 #include <string.h>
4
5 FILE *fp = fopen("missing.txt", "r");
6 if (fp == NULL) {
7     int saved = errno;
8     fprintf(stderr, "open failed: %s\n", strerror(saved));
9 }

```

Listing 66: `errno` pattern

6.5 Low-level file descriptors

A process starts with file descriptors 0, 1, and 2 for standard input, output, and error. Low-level system calls operate on these integers. The standard library's `FILE *` streams are buffered wrappers around lower-level operations.

`open`

- **Context/header:** `##include <fcntl.h>, ##include <sys/stat.h>, ##include <unistd.h>`; man section 2.
- **Form:** `int open(const char *path, int flags, ...);`
- **Purpose:** Opens a file and returns a file descriptor. Common flags include `O_RDONLY`, `O_WRONLY`, `O_CREAT`, `O_TRUNC`, and `O_NONBLOCK`.
- **Result/trap:** Returns -1 on error and sets `errno`. If using `O_CREAT`, provide a mode argument such as `0644`.

`read`

- **Context/header:** `##include <unistd.h>`; man section 2.
- **Form:** `ssize_t read(int fd, void *buf, size_t count);`
- **Purpose:** Reads up to `count` bytes into `buf`.
- **Result/trap:** Returns bytes read, 0 for EOF, -1 on error. It may read fewer bytes than requested. If interrupted by a signal, it can fail with `errno == EINTR`.

`write`

- **Context/header:** `##include <unistd.h>`; man section 2.
- **Form:** `ssize_t write(int fd, const void *buf, size_t count);`

- **Purpose:** Writes up to `count` bytes from `buf`.
- **Result/trap:** Returns bytes written or -1. It can write only part of the buffer, so robust code loops until done.

close

- **Context/header:** `##include <unistd.h>`; man section 2.
- **Form:** `int close(int fd);`
- **Purpose:** Closes a file descriptor.
- **Result/trap:** Returns 0 on success, -1 on error. Closing unused pipe ends is essential for EOF behaviour.

EINTR

- **Context/header:** `##include <errno.h>`.
- **Form:** `if (errno == EINTR) retry_or_handle();`
- **Purpose:** Error code meaning a blocking system call was interrupted by a signal.
- **Result/trap:** Do not blindly treat every -1 as fatal. Decide whether the operation should be retried.

```

1  #include <errno.h>
2  #include <unistd.h>
3
4  int copy_fd(int in, int out) {
5      char buf[4096];
6
7      for (;;) {
8          ssize_t n = read(in, buf, sizeof buf);
9          if (n == 0) {
10             return 0;
11         }
12         if (n < 0) {
13             if (errno == EINTR) {
14                 continue;
15             }
16             return -1;
17         }
18
19         ssize_t off = 0;
20         while (off < n) {
21             ssize_t m = write(out, buf + off, (size_t)(n - off));
22             if (m < 0) {
23                 if (errno == EINTR) {
24                     continue;
25                 }
26                 return -1;
27             }
28             off += m;
29         }
30     }
31 }
```

Listing 67: Low-level cat pattern

6.6 Non-blocking file descriptor flags

`fcntl`

- **Context/header:** `##include <fcntl.h>, ##include <unistd.h>`.
- **Form:** `int fcntl(int fd, int cmd, ...);`
- **Purpose:** Manipulates file descriptor properties. For non-blocking I/O, use `F_GETFL` then `F_SETFL`.
- **Result/trap:** Preserve existing flags when adding `O_NONBLOCK`; do not overwrite them accidentally.

`O_NONBLOCK`

- **Context/header:** `##include <fcntl.h>`.
- **Form:** `open(path, O_RDONLY | O_NONBLOCK);`
- **Purpose:** Makes operations that would block fail immediately with an error such as `EAGAIN` or `EWOULDBLOCK`.
- **Result/trap:** Non-blocking code must handle “not ready yet” as an ordinary state, not a fatal crash.

```
1 int flags = fcntl(fd, F_GETFL);
2 if (flags == -1) {
3     perror("fcntl F_GETFL");
4 }
5 if (fcntl(fd, F_SETFL, flags | O_NONBLOCK) == -1) {
6     perror("fcntl F_SETFL");
7 }
```

Listing 68: Set `O_NONBLOCK` while preserving flags

Common pitfalls

- Misreading function pointer declarations.
- Writing a `qsort` comparator that casts to the wrong element type.
- Doing too much work inside a signal handler.
- Checking `errno` without first checking the function’s return value.
- Assuming `read` and `write` transfer everything in one call.
- Forgetting `EINTR` and partial transfer loops.

6.7 Tutorial Extractions

6.7.1 `qsort` comparator

The tutorial fish-sorting problem is really a function-pointer problem. `qsort` does not know your element type, so it asks for a comparator.

```
1 int cmp_ints(const void *pa, const void *pb) {
2     const int *a = pa;
3     const int *b = pb;
4
5     return (*a > *b) - (*a < *b);
}
```

```
6 }
```

Listing 69: Reusable qsort comparator shape

The expression returns 1, 0, or -1 without overflowing.

6.7.2 Non-blocking stdin pulse

The tutorial non-blocking stdin exercise tests `fcntl`, return values, and `errno`.

```
1 char buf[128];
2 ssize_t n = read(STDIN_FILENO, buf, sizeof buf - 1);
3 if (n > 0) {
4     buf[n] = '\0';
5     printf("You typed: %s\n", buf);
6 } else if (n == -1 && (errno == EAGAIN || errno == EWOULDBLOCK)) {
7     puts("no input yet");
8 } else if (n == 0) {
9     puts("EOF");
10 } else {
11     perror("read");
12 }
```

Listing 70: Non-blocking read decision point

Week takeaway

- Function pointers allow runtime-selected behaviour.
- `sigaction` is the preferred signal interface.
- `errno` must be read only after failure and before it is overwritten.
- Low-level I/O is descriptor-based and requires explicit handling of blocking, interruption, and partial transfers.

6.8 Full-depth expansion pass

Week 6 is the first strong OS-interface week. Function pointers, signals, `errno`, and descriptors all require reading exact contracts. Short code is not automatically simple code.

6.8.1 Teaching-density checklist

Read this week using the original notes standard: each item below should be explainable in words, recognisable in code, and connectable to a failure mode. If an item feels like a vocabulary word only, rewrite it as a small memory, process, descriptor, or thread story.

- function pointer syntax
- callbacks
- qsort comparator contract
- signal disposition
- async-signal-safe handlers
- `errno` after failure

- file descriptors
- partial read
- partial write
- EINTR
- non-blocking flags

6.8.2 Expanded API cards

Expanded API card: `qsort`

Purpose Sort an array using a comparator callback supplied by the caller.

Header `stdlib.h`

Simplified prototype

```
void qsort(void *base, size_t nmemb, size_t size, int (*compar)(const void *,
const void *));
```

Parameter meanings

base points at elements, nmemb counts them, size is each element size, compar compares two element addresses.

Return value No return value.

Failure behaviour

A comparator with inconsistent ordering gives unreliable results.

Trap The comparator receives pointers to elements, not elements.

```
1 int cmp(const void *pa, const void *pb) {
2     const int *a = pa;
3     const int *b = pb;
4     return (*a > *b) - (*a < *b);
5 }
```

Listing 71: Small example for `qsort`

Expanded API card: `sigaction`

Purpose Install a signal disposition with explicit settings.

Header `signal.h`

Simplified prototype

```
int sigaction(int signum, const struct sigaction *act, struct sigaction *oldact);
```

Parameter meanings

signum selects the signal; act describes the handler and mask; oldact may receive the previous action.

Return value Zero on success, minus one on failure.

Failure behaviour

Invalid signals fail; some signals cannot be caught.

Trap Handlers should do tiny async-safe work, often setting a `sig_atomic_t` flag.

```
1 static volatile sig_atomic_t stop;
2 static void h(int s) { (void)s; stop = 1; }
3 struct sigaction sa = {0};
4 sa.sa_handler = h;
5 sigaction(SIGINT, &sa, NULL);
```

Listing 72: Small example for sigaction

Expanded API card: read

Purpose Read up to count bytes from a descriptor.

Header `unistd.h`

Simplified prototype

`ssize_t read(int fd, void *buf, size_t count);`

Parameter meanings

`fd` is open, `buf` is writable, `count` is maximum bytes requested.

Return value Positive byte count, zero for EOF, minus one for error.

Failure behaviour

Can be interrupted or return fewer bytes than requested.

Trap Short read is not automatically an error.

```
1 ssize_t n = read(fd, buf, sizeof buf);
2 if (n == -1 && errno == EINTR) {
3     /* retry */
4 }
```

Listing 73: Small example for read

6.8.3 Tutorial extractions with full reasoning

6.8.4 qsort struct comparator

What the question is testing The task checks callback type and safe comparison.

Required background You need void pointer casts, const correctness, and negative-zero-positive return values.

```
1 int cmp_fish(const void *pa, const void *pb) {
2     const struct fish *a = pa;
3     const struct fish *b = pb;
4     if (a->weight < b->weight) return -1;
5     if (a->weight > b->weight) return 1;
6     return strcmp(a->name, b->name);
7 }
```

Listing 74: Tutorial extraction: qsort struct comparator

Step-by-step reasoning

1. qsort passes addresses of array elements.
2. Cast each void pointer to the correct element pointer type.
3. Compare fields without overflow-prone subtraction.
4. Return a secondary ordering when the primary key ties.

Common mistakes

- Returning true or false.
- Casting to the struct value instead of pointer.
- Modifying elements during comparison.

Takeaway A comparator is a contract, not just any function that compiles.

6.8.5 Robust copy loop

What the question is testing The task checks partial I/O and EINTR.

Required background You need read and write return values, byte offsets, and retry rules.

```
1 ssize_t n;
2 while ((n = read(in, buf, sizeof buf)) != 0) {
3     if (n == -1) {
4         if (errno == EINTR) continue;
5         return -1;
6     }
7     ssize_t off = 0;
8     while (off < n) {
9         ssize_t w = write(out, buf + off, n - off);
10        if (w == -1 && errno == EINTR) continue;
11        if (w == -1) return -1;
12        off += w;
13    }
14 }
```

Listing 75: Tutorial extraction: Robust copy loop

Step-by-step reasoning

1. Read until EOF returns zero.
2. Retry interrupted reads when the operation should continue.
3. For every successful read, write until all n bytes are written.
4. Track an offset because write can be partial.

Common mistakes

- Assuming write writes all bytes.
- Treating EINTR as fatal in every context.
- Ignoring read errors after loop exit.

Takeaway Descriptor I/O is a protocol of byte counts, not a promise of complete buffers.

6.8.6 Exam-readiness prompts

- Before answering, identify the object or resource whose lifetime is being tested.
- State the API success and failure return values before using the result.
- Draw the relevant storage, descriptor table, process tree, or thread-shared invariant.
- Name the common trap in the question and show how the code avoids it.
- Finish with the cleanup or synchronisation step that makes the example complete.

6.8.7 Depth takeaway

This expansion is intentionally slower than a checklist. The final exam rewards the student who can explain why a line is correct, what would break if one return value or close call were missing, and how the same pattern reappears in a later week.

7 Week 7: Compilation, Linking, Preprocessor, Type Safety

7.1 Core idea

C compilation is a pipeline: preprocessing, compilation to assembly, assembly to object code, then linking. Many confusing errors come from mixing these stages. Header files provide declarations; source files provide definitions; the linker resolves external symbols.

7.2 Compilation stages

```
1 gcc -E tasks.c > tasks.i      # preprocessed C
2 gcc -S -g -std=c11 -Wall -Werror list.c
3 gcc -c -g -std=c11 -Wall -Werror list.c
4 gcc main.o list.o -o program
```

Listing 76: Inspecting compilation stages

gcc -c

- **Context/header:** Compiler command.
- **Form:** `gcc -c file.c`
- **Purpose:** Compiles a translation unit into an object file without linking.
- **Result/trap:** The result is usually `file.o`. Undefined external references can remain as placeholders until link time.

linking object files

- **Context/header:** Compiler/linker workflow.
- **Form:** `gcc main.o list.o util.o -o app`
- **Purpose:** Combines object files and libraries into an executable, resolving external symbols.
- **Result/trap:** A symbol should have exactly one definition. Missing definitions cause undefined reference errors; duplicate definitions cause multiple definition errors.

7.3 Headers, declarations, and definitions

A header should declare interfaces. A source file should define storage and functions. Include guards prevent accidental repeated inclusion.

```
1  /* counter.h */
2  #ifndef COUNTER_H
3  #define COUNTER_H
4
5  extern int global_count;
6  void counter_inc(void);
7
8  #endif
9
10 /* counter.c */
11 #include "counter.h"
12
13 int global_count = 0;
14
15 void counter_inc(void) {
16     global_count++;
17 }
```

Listing 77: Header and source split

7.4 Preprocessor

The preprocessor handles directives such as `##include`, `##define`, `##if`, `##ifdef`, `##else`, and `##endif`. These are not ordinary C statements; they are processed before compilation.

```
1  #define BAD_SQUARE(x) x * x
2  #define SQUARE(x) ((x) * (x))
3
4  int a = BAD_SQUARE(1 + 2); /* expands like 1 + 2 * 1 + 2 */
5  int b = SQUARE(1 + 2);    /* expands safely */
```

Listing 78: Macro pitfalls and safer macro style

Macros are text substitution. Use parentheses around macro parameters and around the full macro expression. Prefer functions when possible, especially when arguments have side effects.

7.5 Linkage: extern and static

extern

- **Context/header:** C declaration keyword.
- **Form:** `extern int GLOBAL_VAL;`
- **Purpose:** Declares that a variable or function is defined elsewhere.
- **Result/trap:** A declaration is not a definition. Put one real definition in one source file.

static linkage

- **Context/header:** C storage/linkage keyword.
- **Form:** `static int helper(void);`

- **Purpose:** At file scope, gives internal linkage so the symbol is private to the translation unit.
- **Result/trap:** Good practice for helper functions and file-private globals. Inside a function, `static` gives persistent storage duration instead.

7.6 `const` and `restrict`

`const`

- **Context/header:** C type qualifier.
- **Form:** `const int *p; int * const q = &x;`
- **Purpose:** Communicates and enforces that something should not be modified through that expression.
- **Result/trap:** `const int *p` means pointer to const int; `int * const q` means const pointer to int. Read declarations from the identifier outward.

`restrict`

- **Context/header:** C pointer qualifier.
- **Form:** `void add(int n, int * restrict a, int * restrict b);`
- **Purpose:** Promises that, for the pointer's lifetime, only that pointer is used to access the object. This can enable optimisation.
- **Result/trap:** It is a promise by the programmer. If aliases actually exist, behaviour can be undefined.

7.7 Object inspection tools

The tutorial materials mention `objdump` and `readelf` for inspecting generated code and executable sections.

```
1 objdump -M intel -S list.o
2 readelf --section program
```

Listing 79: Object inspection

Use these to connect C source with assembly, object files, symbols, and memory sections such as `.text`, `.data`, and `.bss`.

Common pitfalls

- Defining global variables in headers.
- Forgetting include guards.
- Treating macros like type-checked functions.
- Misunderstanding `static`: file-private linkage versus persistent local variable.
- Putting linker flags in the wrong place for some libraries.

7.8 Tutorial Extractions

7.8.1 Multi-file linkage

```

1  /* source1.c */
2  int GLOBAL_VAL = 0;
3
4  /* source2.c */
5  extern int GLOBAL_VAL;
6
7  void bump(void) {
8      GLOBAL_VAL++;
9  }

```

Listing 80: One definition, many declarations

The compiler accepts `source2.c` because it knows a variable exists somewhere. The linker must later find exactly one definition.

7.8.2 Tree DFS with dynamic children

The advanced C tutorial includes a tree task. It combines structs, dynamic memory, `realloc`, and recursive traversal.

```

1  struct tree_node {
2      int value;
3      struct tree_node **children;
4      size_t nchildren;
5      size_t capacity;
6  };
7
8  void dfs(const struct tree_node *root) {
9      if (root == NULL) {
10         return;
11     }
12     printf("%d\n", root->value);
13     for (size_t i = 0; i < root->nchildren; i++) {
14         dfs(root->children[i]);
15     }
16 }

```

Listing 81: Tree node with growable child array

The reusable pattern is that `children` is a heap array of pointers. Adding a child may require `realloc`; destroying the tree must recursively destroy children before freeing the array and node.

Week takeaway

- Compilation is staged; know which tool output belongs to which stage.
- Headers declare; source files define.
- Linkage controls symbol visibility across translation units.
- Macros are preprocessor text substitution and need defensive parentheses.
- Type qualifiers such as `const` and `restrict` express important promises.

7.9 Full-depth expansion pass

The expanded Week 7 chapter should read like the missing continuation of the original notes: build stages, preprocessor behaviour, symbol linkage, libraries, and type qualifiers all receive

exam-level treatment.

7.9.1 Teaching-density checklist

Read this week using the original notes standard: each item below should be explainable in words, recognisable in code, and connectable to a failure mode. If an item feels like a vocabulary word only, rewrite it as a small memory, process, descriptor, or thread story.

- gcc -E preprocessing
- gcc -c object files
- headers versus source files
- include guards
- macros as text substitution
- conditional compilation
- external symbols
- static internal linkage
- static libraries
- dynamic libraries
- nm and objdump
- const and restrict contracts

7.9.2 Expanded API cards

Expanded API card: gcc -E

Purpose Run preprocessing only.

Header gcc command

Simplified prototype
gcc -E file.c

Parameter meanings
Input is a source file; output is preprocessed C text.

Return value Preprocessed source on stdout unless redirected.

Failure behaviour
Missing includes or bad directives stop preprocessing.

Trap Expanded output can be huge; search for the macro or declaration in question.

```
1 /* shell */
2 /* gcc -E main.c > main.i */
```

Listing 82: Small example for gcc -E

Expanded API card: gcc -c

Purpose Produce an object file without linking.

Header gcc command

Simplified prototype

`gcc -c file.c -o file.o`

Parameter meanings

Input is one translation unit; -o names object output.

Return value An object file containing code, data, and unresolved references.

Failure behaviour

Undefined external calls can remain unresolved until link time.

Trap

Success here does not prove the final executable will link.

```
1 /* shell */
2 /* gcc -c main.c -o main.o */
3 /* gcc -c list.c -o list.o */
```

Listing 83: Small example for gcc -c

Expanded API card: extern

Purpose Declare that an externally linked object is defined elsewhere.

Header C keyword

Simplified prototype

`extern int global_count;`

Parameter meanings

The declaration gives type information but usually allocates no storage.

Return value No runtime value.

Failure behaviour

If no definition is linked, the linker reports an undefined reference.

Trap

Place extern declarations in headers, but place the one definition in a source file.

```
1 /* counter.h */
2 extern int counter;
3 /* counter.c */
4 int counter = 0;
```

Listing 84: Small example for extern

Expanded API card: static at file scope

Purpose Give a file-scope name internal linkage.

Header C keyword

Simplified prototype

`static int helper(void);`

Parameter meanings

The function or object remains private to that translation unit.

Return value No runtime value.

Failure behaviour

Other files cannot refer to that symbol by external linkage.

Trap At block scope, static means persistent storage duration instead, so context changes meaning.

```
1 static int parse_digit(char c) {  
2     return c - '0';  
3 }
```

Listing 85: Small example for static at file scope

7.9.3 Tutorial extractions with full reasoning

7.9.4 Multi-file undefined reference

What the question is testing The task checks whether the student understands the boundary between successful compilation and failed linking.

Required background You need a header declaration, one implementation definition, object files, and the final link command.

```
1 /* util.h */  
2 int twice(int x);  
3  
4 /* util.c */  
5 int twice(int x) { return x * 2; }  
6  
7 /* main.c */  
8 #include "util.h"  
9 int main(void) { return twice(21); }
```

Listing 86: Tutorial extraction: Multi-file undefined reference

Step-by-step reasoning

1. main.c can compile because util.h declares twice.
2. util.c must also be compiled so the definition exists in util.o.
3. The final link command must include both main.o and util.o.
4. If util.o is omitted, the linker reports undefined reference to twice.

Common mistakes

- Adding the function body to the header and creating duplicates.
- Recompiling only main.c and expecting the linker to find util.c automatically.
- Mistaking a link error for a missing include.

Takeaway Declarations satisfy the compiler; definitions satisfy the linker.

7.9.5 Macro expansion trap

What the question is testing The task checks that macros are text replacement and can duplicate side effects.

Required background You need preprocessor expansion and the difference between macro parameters and function parameters.

```
1 #define MAX(a, b) ((a) > (b) ? (a) : (b))
2
3 int i = 0;
4 int x = MAX(i++, 10);
```

Listing 87: Tutorial extraction: Macro expansion trap

Step-by-step reasoning

1. The preprocessor substitutes `i++` wherever `a` appears.
2. The comparison may increment `i` once.
3. If that branch is selected, the result expression may increment `i` again.
4. A static inline function avoids this double evaluation for typed arguments.

Common mistakes

- Passing expressions with side effects to function-like macros.
- Forgetting parentheses around macro parameters.
- Assuming the macro has a type before expansion.

Takeaway Macro safety requires thinking about the exact expanded text.

7.9.6 Const and restrict API design

What the question is testing The task checks whether qualifiers communicate real contracts.

Required background You need const pointer reading and restrict as an aliasing promise.

```
1 void copy_ints(size_t n, int *restrict dst, const int *restrict src) {
2     for (size_t i = 0; i < n; i++) {
3         dst[i] = src[i];
4     }
5 }
```

Listing 88: Tutorial extraction: Const and restrict API design

Step-by-step reasoning

1. `const` says `copy_ints` will not modify elements through `src`.
2. `restrict` says `dst` and `src` do not alias for the lifetime of those pointer uses.

3. The compiler may optimise based on that promise.
4. If the caller passes overlapping ranges, behaviour can violate the contract.

Common mistakes

- Using restrict casually when aliases are possible.
- Casting away const to write through src.
- Reading `int * const` as pointer to const int.

Takeaway Qualifiers are part of the function contract, not decoration.

7.9.7 Exam-readiness prompts

- Before answering, identify the object or resource whose lifetime is being tested.
- State the API success and failure return values before using the result.
- Draw the relevant storage, descriptor table, process tree, or thread-shared invariant.
- Name the common trap in the question and show how the code avoids it.
- Finish with the cleanup or synchronisation step that makes the example complete.

7.9.8 Depth takeaway

This expansion is intentionally slower than a checklist. The final exam rewards the student who can explain why a line is correct, what would break if one return value or close call were missing, and how the same pattern reappears in a later week.

8 Week 8: Processes, fork, exec, wait, posix_spawn

8.1 Core idea

A process is an executing program with its own virtual memory image, process ID, open file descriptor table, and execution state. `fork` duplicates a process. `exec` replaces the current process image with another program. `wait` and `waitpid` let parents collect child termination status and avoid zombies.

8.2 Process memory image

The process memory image includes program text, initialised data, uninitialised data, heap, memory mappings, stack, environment, and arguments. Processes normally do not share virtual address spaces. After `fork`, parent and child start with similar memory contents, but writes become independent through copy-on-write behaviour.

8.3 fork

fork

- **Context/header:** `##include <unistd.h>`; man section 2.
- **Form:** `pid_t fork(void);`

- **Purpose:** Creates a child process by duplicating the calling process.
- **Result/trap:** Called once but returns twice: child gets 0, parent gets child's PID, failure returns -1 in the parent and sets `errno`. Always branch on the return value.

```

1 pid_t pid = fork();
2 if (pid == -1) {
3     perror("fork");
4     return 1;
5 }
6
7 if (pid == 0) {
8     printf("child\n");
9     _exit(0);
10 }
11
12 printf("parent created child %ld\n", (long)pid);

```

Listing 89: Basic fork branching

After `fork`, both processes continue from the next instruction. Output ordering is not deterministic unless you synchronise.

8.4 exec family

exec family

- **Context/header:** `##include <unistd.h>`; man section 3 in the tutorial.
- **Form:** `execl(path, arg0, ..., NULL); execvp(file, argv);`
- **Purpose:** Replaces the current process image with another program.
- **Result/trap:** On success, it does not return. If it returns, it failed, usually with -1 and `errno` set. Put error handling immediately after the call.

execl

- **Context/header:** `##include <unistd.h>`.
- **Form:** `int execl(const char *path, const char *arg0, ..., NULL);`
- **Purpose:** Execute a program by explicit path with a variadic argument list.
- **Result/trap:** The list must end with `(char *)NULL`. `arg0` conventionally names the program.

execlp

- **Context/header:** `##include <unistd.h>`.
- **Form:** `int execlp(const char *file, const char *arg0, ..., NULL);`
- **Purpose:** Like `execl`, but searches `PATH` when `file` contains no slash.
- **Result/trap:** Convenient for shell-like programs; be aware of `PATH`.

execv

- **Context/header:** `##include <unistd.h>`.
- **Form:** `int execv(const char *path, char *const argv[]);`
- **Purpose:** Execute by explicit path with an argument vector.

- **Result/trap:** The vector must be null-terminated. Useful when arguments are already in an array.

execvp

- **Context/header:** `##include <unistd.h>`.
- **Form:** `int execvp(const char *file, char *const argv[]);`
- **Purpose:** Argument-vector form with PATH search.
- **Result/trap:** Common in mini-shell tutorial patterns. Check failure and call `perror`.

```

1 char *argv[] = {"ls", "-l", NULL};
2
3 pid_t pid = fork();
4 if (pid == -1) {
5     perror("fork");
6     return 1;
7 }
8
9 if (pid == 0) {
10     execvp(argv[0], argv);
11     perror("execvp");
12     _exit(127);
13 }

```

Listing 90: fork plus execvp shell pattern

8.5 Waiting and exit status

wait

- **Context/header:** `##include <sys/wait.h>`.
- **Form:** `pid_t wait(int *status);`
- **Purpose:** Blocks until one child terminates, then reaps it.
- **Result/trap:** Returns child PID, or -1 on error. Use it when any child is acceptable.

waitpid

- **Context/header:** `##include <sys/wait.h>`.
- **Form:** `pid_t waitpid(pid_t pid, int *status, int options);`
- **Purpose:** Waits for a specific child or a class of children.
- **Result/trap:** Can be blocking or non-blocking with `WNOHANG`. Essential when managing multiple children.

WIFEXITED, WEXITSTATUS

- **Context/header:** `##include <sys/wait.h>`.
- **Form:** `WIFEXITED(status); WEXITSTATUS(status);`
- **Purpose:** Inspect whether a child exited normally and retrieve its exit status.
- **Result/trap:** Only call `WEXITSTATUS` if `WIFEXITED(status)` is true.

```

1 int status;
2 pid_t done = waitpid(pid, &status, 0);

```

```

3 if (done == -1) {
4     perror("waitpid");
5 } else if (WIFEXITED(status)) {
6     printf("exit status %d\n", WEXITSTATUS(status));
7 } else if (WIFSIGNALED(status)) {
8     printf("killed by signal %d\n", WTERMSIG(status));
9 }

```

Listing 91: waitpid status handling

8.6 Zombies and fork bombs

A zombie is a terminated child that still has an entry in the process table because the parent has not reaped it. A fork bomb is uncontrolled repeated forking, often exponential.

```

1 for (int i = 0; i < n; i++) {
2     fork();          /* bad: can create 2^n processes */
3 }

```

Listing 92: Fork bomb shape to recognise, not copy

Safe process-creation code checks `fork`, branches immediately, and waits in the parent when appropriate.

8.7 `posix_spawn`

`posix_spawn`

- **Context/header:** `##include <spawn.h>` plus environment declaration; POSIX.
- **Form:** `int posix_spawn(pid_t *pid, const char *path, const posix_spawn_file_actions_t *file_actions, const posix_spawnattr_t *attrp, char *const argv[], char *const envp[]);`
- **Purpose:** Creates a new process running a chosen program, combining much of `fork+exec` into one call.
- **Result/trap:** Returns 0 on success or an error number on failure. Use when you want explicit `argv` and optional file actions without manually composing all `fork/exec` details.

```

1 #include <spawn.h>
2 #include <sys/wait.h>
3
4 extern char **environ;
5
6 pid_t pid;
7 char *argv[] = {"ls", "-l", NULL};
8 int rc = posix_spawn(&pid, "/bin/ls", NULL, NULL, argv, environ);
9 if (rc != 0) {
10     errno = rc;
11     perror("posix_spawn");
12     return 1;
13 }
14 waitpid(pid, NULL, 0);

```

Listing 93: Basic `posix_spawn` pattern

Common pitfalls

- Forgetting that `fork` returns in both parent and child.
- Assuming parent and child output order is deterministic.
- Writing code after successful `exec` and expecting it to run.
- Failing to `wait`, producing zombies.
- Calling `WEXITSTATUS` without first checking `WIFEXITED`.

8.8 Tutorial Extractions

8.8.1 Fork output counting

If a program calls `fork` twice without branching, the number of processes doubles each time.

```

1 printf("A\n");
2 fork();
3 printf("B\n");
4 fork();
5 printf("C\n");

```

Listing 94: Counting forked processes

Reasoning:

- One process prints A.
- After first `fork`, two processes print B.
- After second `fork`, four processes print C.
- The order is scheduler-dependent.

8.8.2 Mini-shell pattern

```

1 pid_t pid = fork();
2 if (pid == -1) {
3     perror("fork");
4 } else if (pid == 0) {
5     execvp(argv[0], argv);
6     perror("execvp");
7     _exit(127);
8 } else {
9     int status;
10    if (waitpid(pid, &status, 0) == -1) {
11        perror("waitpid");
12    }
13 }

```

Listing 95: Execute one command and wait

The reusable idea is that the child becomes the command, while the parent remains the shell and waits.

Week takeaway

- Processes have separate memory images.
- `fork` duplicates, `exec` replaces, `waitpid` reaps and reports status.

- Successful `exec` does not return.
- Use `posix_spawn` for a more structured spawn interface when it fits the task.

8.9 Full-depth expansion pass

The expanded process chapter should be read as an OS story. `fork` duplicates execution, `exec` replaces the image, `wait` reaps children, and `posix_spawn` packages the common launch pattern.

8.9.1 Teaching-density checklist

Read this week using the original notes standard: each item below should be explainable in words, recognisable in code, and connectable to a failure mode. If an item feels like a vocabulary word only, rewrite it as a small memory, process, descriptor, or thread story.

- process memory image
- virtual address space
- `fork` returns twice
- parent versus child branch
- copy-on-write intuition
- `exec` does not return on success
- `argv` vectors
- `PATH` search
- `wait` status macros
- zombies
- `fork` bombs
- `posix_spawn` file actions

8.9.2 Expanded API cards

Expanded API card: `fork`

Purpose Create a child process duplicating the caller.

Header `unistd.h`

Simplified prototype
`pid_t fork(void);`

Parameter meanings
 No arguments. The return value distinguishes parent and child.

Return value Zero in the child, child PID in the parent, minus one on failure.

Failure behaviour
 Resource limits can make `fork` fail.

Trap Every process that reaches code after `fork` continues unless branched away.

```

1 pid_t pid = fork();
2 if (pid == 0) {
3     _exit(0);
4 } else if (pid > 0) {
5     waitpid(pid, NULL, 0);
6 }

```

Listing 96: Small example for fork

Expanded API card: `execvp`

Purpose Replace the current process image using PATH search.

Header `unistd.h`

Simplified prototype

```
int execvp(const char *file, char *const argv[]);
```

Parameter meanings

file is command name or path; argv is a NULL-terminated argument vector.

Return value Does not return on success; returns -1 on failure.

Failure behaviour

Missing command or permission errors set `errno`.

Trap

Code after `execvp` is failure handling only.

```

1 char *av[] = { "ls", "-l", NULL };
2 execvp(av[0], av);
3 perror("execvp");
4 _exit(127);

```

Listing 97: Small example for `execvp`

Expanded API card: `waitpid`

Purpose Wait for a selected child and reap its status.

Header `sys/wait.h`

Simplified prototype

```
pid_t waitpid(pid_t pid, int *status, int options);
```

Parameter meanings

pid selects child; status receives encoded status; options may include `WNOHANG`.

Return value Child PID, zero with `WNOHANG` and no change, or -1.

Failure behaviour

Can fail or be interrupted by signals.

Trap

Use `WIFEXITED` before `WEXITSTATUS`.

```

1 int st;
2 if (waitpid(pid, &st, 0) == pid && WIFEXITED(st)) {
3     printf("%d\n", WEXITSTATUS(st));
4 }

```

Listing 98: Small example for waitpid

Expanded API card: `posix_spawn`

Purpose Create a process running a new program with optional actions.

Header `spawn.h`

Simplified prototype

```
int posix_spawn(pid_t *pid, const char *path, const posix_spawn_file_actions_t
*fa, const posix_spawnattr_t *attr, char *const argv[], char *const envp[]);
```

Parameter meanings

`pid` receives child ID; `path` and `argv` describe program; file actions describe descriptor changes.

Return value Zero on success, otherwise an error number.

Failure behaviour

The return value itself is the error code; do not blindly inspect `errno`.

Trap `argv` and `envp` still need NULL termination.

```

1 char *av[] = { "echo", "hi", NULL };
2 pid_t pid;
3 int err = posix_spawn(&pid, "/bin/echo", NULL, NULL, av, environ);

```

Listing 99: Small example for `posix_spawn`

8.9.3 Tutorial extractions with full reasoning

8.9.4 Fork output count

What the question is testing The task checks process multiplication, not line ordering.

Required background You need to count how many processes reach each fork and remember scheduling is unconstrained.

```

1 printf("A\n");
2 fork();
3 printf("B\n");
4 fork();
5 printf("C\n");

```

Listing 100: Tutorial extraction: Fork output count

Step-by-step reasoning

1. One process prints A.
2. The first fork makes two processes.
3. Both print B.
4. Both reach the second fork, making four processes.
5. Four processes print C in unspecified order.

Common mistakes

- Assuming parent output always comes first.
- Counting fork calls rather than processes reaching fork.
- Forgetting output buffering when redirected.

Takeaway For fork questions, track live processes after each fork.

8.9.5 Mini-shell command

What the question is testing The task checks fork-exec-wait composition.

Required background You need to keep the shell in the parent and run the command in the child.

```
1 pid_t pid = fork();
2 if (pid == 0) {
3     execvp(argv[0], argv);
4     perror(argv[0]);
5     _exit(127);
6 }
7 waitpid(pid, NULL, 0);
```

Listing 101: Tutorial extraction: Mini-shell command

Step-by-step reasoning

1. Fork so the parent shell remains alive.
2. In the child, exec the command.
3. If exec returns, report failure and exit only the child.
4. In the parent, wait for the specific child.

Common mistakes

- Calling exec in the parent.
- Forgetting NULL at the end of argv.
- Letting the child fall back into shell code after exec failure.

Takeaway The child transforms; the parent supervises.

8.9.6 Exam-readiness prompts

- Before answering, identify the object or resource whose lifetime is being tested.
- State the API success and failure return values before using the result.
- Draw the relevant storage, descriptor table, process tree, or thread-shared invariant.
- Name the common trap in the question and show how the code avoids it.
- Finish with the cleanup or synchronisation step that makes the example complete.

8.9.7 Depth takeaway

This expansion is intentionally slower than a checklist. The final exam rewards the student who can explain why a line is correct, what would break if one return value or close call were missing, and how the same pattern reappears in a later week.

9 Week 9: IPC, Pipes, Redirection, popen/pclose

9.1 Core idea

Inter-process communication lets processes cooperate. The course focuses on file descriptors, pipes, redirection, and stream wrappers. A pipe is a kernel-buffered one-way byte channel with a read end and a write end.

9.2 pipe

pipe

- **Context/header:** `##include <unistd.h>`; man section 2.
- **Form:** `int pipe(int pipefd[2]);`
- **Purpose:** Creates two file descriptors: `pipefd[0]` for reading and `pipefd[1]` for writing.
- **Result/trap:** Returns 0 on success, -1 on failure. Close unused ends in each process or EOF and blocking behaviour will be wrong.

```
1 int pfd[2];
2 if (pipe(pfd) == -1) {
3     perror("pipe");
4     return 1;
5 }
6
7 pid_t pid = fork();
8 if (pid == 0) {
9     close(pfd[1]);          /* child does not write */
10    char buf[128];
11    ssize_t n = read(pfd[0], buf, sizeof buf);
12    if (n > 0) {
13        write(STDOUT_FILENO, buf, (size_t)n);
14    }
15    close(pfd[0]);
16    _exit(0);
17 }
18
19 close(pfd[0]);             /* parent does not read */
20 write(pfd[1], "hello\n", 6);
```

```

21 close(pfd[1]);           /* lets child see EOF */
22 waitpid(pid, NULL, 0);

```

Listing 102: Parent writes, child reads

9.3 Pipe EOF and unused ends

A read from a pipe returns 0 only when all write ends referring to that pipe are closed. This includes duplicated descriptors and descriptors inherited across **fork**. If the parent forgets to close its write end, a child reader can block forever waiting for more data.

9.4 dup, dup2, fdopen, fileno

dup

- **Context/header:** `##include <unistd.h>`.
- **Form:** `int dup(int oldfd);`
- **Purpose:** Duplicates a file descriptor into the lowest-numbered unused descriptor slot.
- **Result/trap:** Returns the new descriptor or -1. Useful for understanding descriptor table manipulation.

dup2

- **Context/header:** `##include <unistd.h>`.
- **Form:** `int dup2(int oldfd, int newfd);`
- **Purpose:** Duplicates `oldfd` so it becomes exactly `newfd`, closing `newfd` first if necessary.
- **Result/trap:** Used for redirection before **exec**. Check failure; after successful `dup2`, close the original descriptor if no longer needed.

fdopen

- **Context/header:** `##include <stdio.h>`.
- **Form:** `FILE *fdopen(int fd, const char *mode);`
- **Purpose:** Wraps an existing file descriptor in a `FILE *` stream.
- **Result/trap:** The stream owns the descriptor; `fclose` will close it. Useful when you want `fgets` or `fscanf` on a pipe.

fileno

- **Context/header:** `##include <stdio.h>`.
- **Form:** `int fileno(FILE *stream);`
- **Purpose:** Gets the underlying file descriptor from a `FILE *` stream.
- **Result/trap:** Do not mix buffered stream operations and raw descriptor operations casually; buffering can surprise you.

```

1 int pfd[2];
2 pipe(pfd);
3
4 pid_t pid = fork();
5 if (pid == 0) {

```

```

6     close(pfd[0]);
7     if (dup2(pfd[1], STDOUT_FILENO) == -1) {
8         _exit(127);
9     }
10    close(pfd[1]);
11    execlp("ls", "ls", "-l", (char *)NULL);
12    _exit(127);
13 }
14
15 close(pfd[1]);
16 FILE *from_child = fdopen(pfd[0], "r");
17 char line[256];
18 while (fgets(line, sizeof line, from_child) != NULL) {
19     fputs(line, stdout);
20 }
21 fclose(from_child);
22 waitpid(pid, NULL, 0);

```

Listing 103: Redirect child stdout into a pipe

9.5 popen and pclose

popen

- **Context/header:** `#include <stdio.h>`; man section 3.
- **Form:** `FILE *popen(const char *command, const char *type);`
- **Purpose:** Runs a shell command and returns a stream connected to the command's stdout for "r" or stdin for "w".
- **Result/trap:** Returns NULL on failure. It uses the shell, so command strings need care. It is one-way, not both reading and writing.

pclose

- **Context/header:** `include <stdio.h>`; man section 3.
- **Form:** `int pclose(FILE *stream);`
- **Purpose:** Closes a popen stream and waits for the command to terminate.
- **Result/trap:** Returns a wait-style status. Inspect it with wait macros when needed. Do not use fclose on a popen stream.

```

1 FILE *cmd = popen("printf 'alpha\\nbeta\\n'", "r");
2 if (cmd == NULL) {
3     perror("popen");
4     return 1;
5 }
6
7 char line[128];
8 while (fgets(line, sizeof line, cmd) != NULL) {
9     fputs(line, stdout);
10 }
11
12 int status = pclose(cmd);
13 if (status == -1) {
14     perror("pclose");
15 }

```

9.6 posix_spawn file actions

The IPC lecture shows `posix_spawn` with file actions, such as closing a pipe end in the spawned process. This is the structured alternative to doing `fork`, manual `dup2`, and `exec` yourself.

```
1 posix_spawn_file_actions_t actions;
2 posix_spawn_file_actions_init(&actions);
3 posix_spawn_file_actions_addclose(&actions, pfd[0]);
4 /* add dup2 actions here when redirecting */
5 posix_spawn_file_actions_destroy(&actions);
```

Listing 105: `posix_spawn` file action idea

Common pitfalls

- Forgetting that one pipe is one-way; two-way conversation usually needs two pipes.
- Leaving unused pipe ends open and preventing EOF.
- Redirecting after `exec`; redirection must happen before the child image is replaced.
- Using `fclose` instead of `pclose` for `popen`.
- Mixing `FILE *` buffering and raw file descriptor reads/writes without a plan.

9.7 Tutorial Extractions

9.7.1 Parent-child request/response

The “COMP2017 time” tutorial pattern uses two pipes to force a conversation order.

```
1 int parent_to_child[2];
2 int child_to_parent[2];
3 pipe(parent_to_child);
4 pipe(child_to_parent);
5
6 pid_t pid = fork();
7 if (pid == 0) {
8     close(parent_to_child[1]);
9     close(child_to_parent[0]);
10    /* read request from parent_to_child[0] */
11    /* write response to child_to_parent[1] */
12    _exit(0);
13 }
14
15 close(parent_to_child[0]);
16 close(child_to_parent[1]);
17 /* write request, then read response */
```

Listing 106: Two-pipe conversation sketch

The reusable idea is ownership of pipe ends. Draw which process reads and which process writes before coding.

9.7.2 Mini-shell pipeline

For `ls -al | grep -i Ke`, the first child redirects stdout to the pipe write end; the second redirects stdin from the pipe read end. The parent closes both ends and waits for both children.

```
1 if (left_child == 0) {
2     dup2(pfd[1], STDOUT_FILENO);
3     close(pfd[0]);
4     close(pfd[1]);
5     execvp(left_argv[0], left_argv);
6     _exit(127);
7 }
8
9 if (right_child == 0) {
10     dup2(pfd[0], STDIN_FILENO);
11     close(pfd[0]);
12     close(pfd[1]);
13     execvp(right_argv[0], right_argv);
14     _exit(127);
15 }
```

Listing 107: Pipeline redirection core

Week takeaway

- Pipes are descriptor pairs: read end 0, write end 1.
- EOF depends on closing every write end.
- `dup2` is the key operation behind shell redirection.
- `popen/pclose` are convenient stream wrappers for simple one-way command I/O.

9.8 Full-depth expansion pass

The expanded IPC chapter should make descriptor flow visible. A pipe bug is usually not mysterious; it is an unclosed descriptor end, a wrong `dup2` target, or a blocking assumption that was never made true.

9.8.1 Teaching-density checklist

Read this week using the original notes standard: each item below should be explainable in words, recognisable in code, and connectable to a failure mode. If an item feels like a vocabulary word only, rewrite it as a small memory, process, descriptor, or thread story.

- message passing
- anonymous pipes
- named pipes
- descriptor inheritance
- close unused ends
- EOF semantics
- `dup` and `dup2`
- `fdopen` and buffering
- `popen` one-way streams

- mmap shared memory
- select readiness
- poll revents
- epoll registration

9.8.2 Expanded API cards

Expanded API card: pipe

Purpose	Create a unidirectional kernel byte buffer.
Header	unistd.h
Simplified prototype	int pipe(int pipefd[2]);
Parameter meanings	pipefd[0] becomes read end; pipefd[1] becomes write end.
Return value	Zero on success, minus one on failure.
Failure behaviour	Descriptor or memory limits can fail.
Trap	EOF requires all write ends in all processes to be closed.

```

1 int p[2];
2 if (pipe(p) == -1) perror("pipe");

```

Listing 108: Small example for pipe

Expanded API card: dup2

Purpose	Make newfd refer to the same open file description as oldfd.
Header	unistd.h
Simplified prototype	int dup2(int oldfd, int newfd);
Parameter meanings	oldfd must be valid; newfd is the target descriptor number.
Return value	newfd on success, minus one on failure.
Failure behaviour	Invalid oldfd fails; newfd is closed first if open.
Trap	After redirection, close the original descriptor if no longer needed.

```

1 dup2(p[1], STDOUT_FILENO);
2 close(p[1]);

```

Listing 109: Small example for dup2

Expanded API card: popen

Purpose Run a shell command with one connected stream.

Header `stdio.h`

Simplified prototype

`FILE *popen(const char *command, const char *type);`

Parameter meanings

`command` is interpreted by the shell; `type` is `r` or `w`.

Return value `FILE` pointer or `NULL`.

Failure behaviour

The child command can still fail after `popen` succeeds.

Trap Use `pclose`, not `fclose`, and beware shell injection.

```
1 FILE *fp = popen("wc -l", "w");
2 fprintf(fp, "a\nb\n");
3 pclose(fp);
```

Listing 110: Small example for `popen`

Expanded API card: select

Purpose Wait until descriptors are ready for I/O.

Header `sys/select.h`

Simplified prototype

`int select(int nfds, fd_set *r, fd_set *w, fd_set *e, struct timeval *t);`

Parameter meanings

`nfds` is max descriptor plus one; sets describe interests; timeout controls waiting.

Return value Ready count, zero timeout, or minus one.

Failure behaviour

Sets are modified by the call.

Trap Rebuild `fd_set` values before every `select`.

```
1 FD_ZERO(&set);
2 FD_SET(fd, &set);
3 select(fd + 1, &set, NULL, NULL, NULL);
```

Listing 111: Small example for `select`

Expanded API card: mmap

Purpose Map a file or shared memory object into address space.

Header sys/mman.h

Simplified prototype

```
void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset);
```

Parameter meanings

prot sets permissions; flags choose shared or private; fd and offset name backing storage.

Return value Mapped address or MAP_FAILED.

Failure behaviour

Bad permissions, offsets, or lengths fail.

Trap Compare with MAP_FAILED, not NULL.

```
1 void *p = mmap(NULL, len, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
2 if (p == MAP_FAILED) perror("mmap");
```

Listing 112: Small example for mmap

9.8.3 Tutorial extractions with full reasoning

9.8.4 Parent-child two-pipe exchange

What the question is testing The task checks that a pipe is one-way.

Required background You need two pipes, close discipline, and a simple protocol.

```
1 int req[2], res[2];
2 pipe(req);
3 pipe(res);
4 pid_t pid = fork();
5 if (pid == 0) {
6     close(req[1]); close(res[0]);
7     /* read req[0], write res[1] */
8     _exit(0);
9 }
10 close(req[0]); close(res[1]);
```

Listing 113: Tutorial extraction: Parent-child two-pipe exchange

Step-by-step reasoning

1. Create both pipes before fork.
2. Child closes parent-only ends.
3. Parent closes child-only ends.
4. The request pipe carries parent to child.
5. The response pipe carries child to parent.

Common mistakes

- Using one pipe for both directions without a protocol.
- Leaving write ends open and losing EOF.
- Ignoring partial reads and writes.

Takeaway Draw descriptor ownership before writing pipe code.

9.8.5 Pipeline redirection

What the question is testing The task checks descriptor remapping before exec.

Required background You need pipe, fork twice, dup2, close, exec, and wait.

```
1 int p[2];
2 pipe(p);
3 if (fork() == 0) {
4     dup2(p[1], STDOUT_FILENO);
5     close(p[0]); close(p[1]);
6     execvp(left[0], left);
7     _exit(127);
8 }
9 if (fork() == 0) {
10    dup2(p[0], STDIN_FILENO);
11    close(p[0]); close(p[1]);
12    execvp(right[0], right);
13    _exit(127);
14 }
15 close(p[0]); close(p[1]);
```

Listing 114: Tutorial extraction: Pipeline redirection

Step-by-step reasoning

1. Left child sends stdout into pipe write end.
2. Right child receives stdin from pipe read end.
3. Both children close unused original pipe descriptors.
4. The parent closes both ends so EOF can propagate.
5. The parent waits for both children.

Common mistakes

- Doing dup2 after exec.
- Forgetting to close parent pipe ends.
- Waiting for only one child.

Takeaway A shell pipeline is descriptor preparation followed by exec.

9.8.6 Readiness API comparison

What the question is testing The task checks that select, poll, and epoll report readiness, not completed I/O.

Required background You need fd_set rebuilding, poll revents, and epoll registration.

```
1 /* select modifies sets */
2 /* poll reports through revents */
3 /* epoll registers interest then waits for events */
```

Listing 115: Tutorial extraction: Readiness API comparison

Step-by-step reasoning

1. select requires maxfd plus one.
2. select fd_sets must be rebuilt every loop.
3. poll keeps interests in events and results in revents.
4. epoll stores interest in a kernel object and returns event records.
5. After readiness, still call read or write and handle short transfers.

Common mistakes

- Treating readiness as an entire message.
- Checking poll events instead of revents.
- Forgetting epoll is Linux-specific.

Takeaway Readiness means an operation should not block now; it is not a replacement for I/O handling.

9.8.7 Exam-readiness prompts

- Before answering, identify the object or resource whose lifetime is being tested.
- State the API success and failure return values before using the result.
- Draw the relevant storage, descriptor table, process tree, or thread-shared invariant.
- Name the common trap in the question and show how the code avoids it.
- Finish with the cleanup or synchronisation step that makes the example complete.

9.8.8 Depth takeaway

This expansion is intentionally slower than a checklist. The final exam rewards the student who can explain why a line is correct, what would break if one return value or close call were missing, and how the same pattern reappears in a later week.

9.9 Extended IPC: mmap, select, poll, epoll, Non-blocking I/O

9.10 Core idea

Blocking I/O can stop a process at one descriptor while useful work is waiting elsewhere. Non-blocking I/O and readiness APIs let a program decide when a descriptor can be read or written without getting stuck. Memory mapping gives another communication style: map bytes into an address space and access them like memory.

9.11 mmap and munmap

mmap

- **Context/header:** `##include <sys/mman.h>`.
- **Form:** `void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset);`
- **Purpose:** Creates a memory mapping: file-backed, shared memory, private anonymous memory, or other variants depending on flags.
- **Result/trap:** Returns `MAP_FAILED` on error and sets `errno`. The mapping length and offset matter. Access permissions must match `prot`.

munmap

- **Context/header:** `##include <sys/mman.h>`.
- **Form:** `int munmap(void *addr, size_t length);`
- **Purpose:** Removes a mapping created by `mmap`.
- **Result/trap:** Returns 0 on success or -1. After unmapping, pointers into the region are invalid.

MAP_SHARED, MAP_PRIVATE

- **Context/header:** `##include <sys/mman.h>`.
- **Form:** `mmap(NULL, len, prot, MAP_SHARED, fd, 0);`
- **Purpose:** `MAP_SHARED` makes updates visible through the shared mapping/file as defined by the OS; `MAP_PRIVATE` gives copy-on-write private changes.
- **Result/trap:** Choose based on whether processes should communicate through the mapped region.

PROT_READ, PROT_WRITE

- **Context/header:** `##include <sys/mman.h>`.
- **Form:** `PROT_READ | PROT_WRITE`
- **Purpose:** Mapping permissions: readable, writable, executable, or none.
- **Result/trap:** Accessing memory without the needed permission causes a fault. The tutorial discusses `PROT_NONE` as a way to reserve inaccessible address space.

```
1 int fd = open(path, O_RDONLY);
2 if (fd == -1) {
3     perror("open");
4     return 1;
5 }
```

```

6
7 struct stat st;
8 if (fstat(fd, &st) == -1) {
9     perror("fstat");
10    close(fd);
11    return 1;
12 }
13
14 char *mem = mmap(NULL, (size_t)st.st_size, PROT_READ, MAP_PRIVATE, fd, 0);
15 if (mem == MAP_FAILED) {
16     perror("mmap");
17     close(fd);
18     return 1;
19 }
20
21 /* scan mem[0] through mem[st.st_size - 1] */
22
23 munmap(mem, (size_t)st.st_size);
24 close(fd);

```

Listing 116: Map a file for scanning

9.12 select

select

- **Context/header:** `##include <sys/select.h>`.
- **Form:** `int select(int nfds, fd_set *readfds, fd_set *writefds, fd_set *exceptfds, struct timeval *timeout);`
- **Purpose:** Waits until one or more descriptors become ready for reading, writing, or exceptional conditions.
- **Result/trap:** The first argument is highest descriptor plus one. The sets are modified by the call, so rebuild them before each use.

FD_ZERO, FD_SET, FD_ISSET

- **Context/header:** `##include <sys/select.h>`.
- **Form:** `FD_ZERO(&set); FD_SET(fd, &set); FD_ISSET(fd, &set);`
- **Purpose:** Macros for building and inspecting descriptor sets for `select`.
- **Result/trap:** `select` has scalability and descriptor-limit issues. It is still important because it teaches readiness.

```

1 fd_set readfds;
2 FD_ZERO(&readfds);
3 FD_SET(fd1, &readfds);
4 FD_SET(fd2, &readfds);
5
6 int maxfd = (fd1 > fd2) ? fd1 : fd2;
7 int nready = select(maxfd + 1, &readfds, NULL, NULL, NULL);
8 if (nready == -1) {
9     perror("select");
10 } else {
11     if (FD_ISSET(fd1, &readfds)) {
12         /* fd1 readable */
13     }

```

```

14     if (FD_ISSET(fd2, &readfds)) {
15         /* fd2 readable */
16     }
17 }

```

Listing 117: select readiness pattern

9.13 poll

poll

- **Context/header:** `##include <poll.h>`.
- **Form:** `int poll(struct pollfd fds[], nfds_t nfds, int timeout);`
- **Purpose:** Waits for readiness events on an array of descriptors.
- **Result/trap:** Returns number of descriptors with events, 0 on timeout, -1 on error. Check `revents`, not `events`, after the call.

```

1 struct pollfd fds[2] = {
2     {.fd = pipe1, .events = POLLIN},
3     {.fd = pipe2, .events = POLLIN}
4 };
5
6 int n = poll(fds, 2, -1);
7 if (n == -1) {
8     perror("poll");
9 }
10 for (size_t i = 0; i < 2; i++) {
11     if (fds[i].revents & POLLIN) {
12         /* read fds[i].fd */
13     }
14 }

```

Listing 118: poll for several child pipes

9.14 epoll

epoll

- **Context/header:** Linux-specific readiness interface, `##include <sys/epoll.h>`.
- **Form:** `epoll_create1`, `epoll_ctl`, `epoll_wait`
- **Purpose:** Registers interest in descriptor events with the kernel and later waits for active events.
- **Result/trap:** More scalable than repeatedly scanning all descriptors. Non-blocking descriptors are usually used in event loops.

```

1 int epfd = epoll_create1(0);
2 if (epfd == -1) {
3     perror("epoll_create1");
4 }
5
6 struct epoll_event ev = {0};
7 ev.events = EPOLLIN;
8 ev.data.fd = fd;
9 if (epoll_ctl(epfd, EPOLL_CTL_ADD, fd, &ev) == -1) {

```

```

10     perror("epoll_ctl");
11 }
12
13 struct epoll_event events[16];
14 int n = epoll_wait(epfd, events, 16, -1);
15 if (n == -1) {
16     perror("epoll_wait");
17 }

```

Listing 119: epoll core pattern

9.15 Blocking versus non-blocking design

Blocking reads are simple but can stall the whole program. Non-blocking descriptors return immediately when no data is available. Readiness APIs sit between those extremes: block until at least one descriptor is useful, then act only on ready descriptors.

Common pitfalls

- Treating “ready” as “the whole message is available”.
- Forgetting to rebuild `fd_sets` before each `select`.
- Checking `pollfd.events` instead of `pollfd.revents`.
- Using blocking reads inside an event loop and accidentally hanging the loop.
- Forgetting to `munmap` and `close`.

9.16 Tutorial Extractions

9.16.1 mmap word scanning

The word-count tutorial pattern maps a large file so scanning is pointer arithmetic over memory instead of repeated buffered reads.

```

1 size_t count = 0;
2 for (off_t i = 0; i + needle_len <= st.st_size; i++) {
3     if (memcmp(mem + i, needle, needle_len) == 0) {
4         count++;
5     }
6 }

```

Listing 120: Scanning a mapped range

The reusable idea is that `mmap` does not give a C string. It gives bytes. Use the known file size rather than `strlen`.

9.16.2 Monitoring many children

The IPC tutorial asks for one parent to monitor several child pipes without one thread per pipe. The pattern is:

1. Create one pipe per child.
2. In each child, use `dup2` so `stdout/stderr` point to the pipe write end.

3. In the parent, close write ends and use `poll` or `epoll` on all read ends.
4. Read only the descriptors reported ready.

Week takeaway

- `mmap` maps bytes into memory; it does not create strings or structs by magic.
- `select`, `poll`, and `epoll` are readiness mechanisms.
- Non-blocking I/O turns “would block” into a state your program handles.
- Event loops must avoid accidental blocking inside readiness handling.

10 Week 10: Parallelism and POSIX Threads

10.1 Full-depth expansion pass

The expanded Week 10 chapter should introduce threads as shared-address-space execution, then slow down around lifecycle and argument passing. Most beginner pthread bugs are pointer bugs wearing a concurrency jacket.

10.1.1 Teaching-density checklist

Read this week using the original notes standard: each item below should be explainable in words, recognisable in code, and connectable to a failure mode. If an item feels like a vocabulary word only, rewrite it as a small memory, process, descriptor, or thread story.

- threads versus processes
- shared virtual address space
- thread stacks
- `pthread_create` lifecycle
- `pthread_join` cleanup
- void pointer argument
- argument lifetime
- task parallelism
- data parallelism
- dependencies
- Amdahl law
- ThreadSanitizer

10.1.2 Expanded API cards

Expanded API card: `pthread_create`

Purpose Create a new thread that begins by calling `start(arg)`.

Header `pthread.h`

Simplified prototype

```
int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start)(void *), void *arg);
```


Parameter meanings

thread receives ID; attr is often NULL; start is function pointer; arg is one pointer-sized argument.

Return value Zero on success, otherwise an error number.

Failure behaviour

Resource limits or invalid attributes can fail.

Trap The argument object must stay alive long enough for the worker to use it.

```
1 pthread_t tid;
2 struct job job = {0};
3 pthread_create(&tid, NULL, worker, &job);
```

Listing 121: Small example for pthread_create

Expanded API card: pthread_join

Purpose Wait for a joinable thread and collect its result.

Header pthread.h

Simplified prototype

int pthread_join(pthread_t thread, void **retval);

Parameter meanings

thread names the target; retval may receive the returned pointer.

Return value Zero on success, otherwise an error number.

Failure behaviour

Joining twice or joining a detached thread is invalid.

Trap Join is resource cleanup as well as ordering.

```
1 void *ret;
2 pthread_join(tid, &ret);
```

Listing 122: Small example for pthread_join

Expanded API card: pthread_exit

Purpose End the calling thread with a return value.

Header pthread.h

Simplified prototype

void pthread_exit(void *retval);

Parameter meanings

retval is available to a joining thread.

Return value Does not return.

Failure behaviour

If used in main, other threads can continue running.

Trap

Returning from the start routine is usually simpler.

```

1 void *worker(void *arg) {
2     return arg;
3 }

```

Listing 123: Small example for pthread_exit

Expanded API card: -pthread**Purpose**

Compile and link with POSIX thread support.

Header

compiler flag

Simplified prototype

gcc file.c -o file -pthread

Parameter meanings

The flag informs both compilation and linking.

Return value

An executable linked with pthread support.

Failure behaviour

Missing the flag can cause link or runtime support problems.

Trap

Prefer -pthread over manually writing only -lpthread.

```

1 /* gcc -Wall -Wextra -g main.c -o main -pthread */

```

Listing 124: Small example for -pthread

10.1.3 Tutorial extractions with full reasoning**10.1.4 Safe thread arguments**

What the question is testing The task checks object lifetime and uniqueness for each worker.

Required background You need an array of argument structs that remains alive until after joins.

```

1 struct arg { int id; size_t start; size_t end; };
2 struct arg args[N];
3 pthread_t tids[N];
4 for (int i = 0; i < N; i++) {
5     args[i].id = i;
6     pthread_create(&tids[i], NULL, worker, &args[i]);
7 }
8 for (int i = 0; i < N; i++) pthread_join(tids[i], NULL);

```

Listing 125: Tutorial extraction: Safe thread arguments

Step-by-step reasoning

1. Allocate one argument object per thread.
2. Fill it before creation.
3. Pass the address of that unique object.
4. Do not let the array go out of scope until all joins complete.

Common mistakes

- Passing `&i` from the loop.
- Using a stack object that disappears before the worker reads it.
- Casting `arg` to the wrong type.

Takeaway Thread arguments are still pointer lifetime questions.

10.1.5 Parallel sum without shared writes

What the question is testing The task checks designing away a race.

Required background You need partitioning, per-thread local total, and final reduction after joins.

```
1 void *sum_worker(void *ptr) {  
2     struct job *j = ptr;  
3     long total = 0;  
4     for (size_t i = j->start; i < j->end; i++) total += j->xs[i];  
5     j->result = total;  
6     return NULL;  
7 }
```

Listing 126: Tutorial extraction: Parallel sum without shared writes

Step-by-step reasoning

1. Split the array into non-overlapping ranges.
2. Each thread writes only its own result field.
3. No shared counter is updated inside the loop.
4. Join before reading result fields.
5. Add the partial results in one thread.

Common mistakes

- Having every worker increment one global sum.
- Reading results before join.
- Creating empty or overlapping chunks.

Takeaway Often the best synchronisation is avoiding shared mutation in the hot path.

10.1.6 Amdahl speedup limit

What the question is testing The task checks whether students understand serial bottlenecks.

Required background You need serial fraction, parallel fraction, and worker count.

```
1 double serial = 0.25;
2 double parallel = 0.75;
3 double workers = 8.0;
4 double speedup = 1.0 / (serial + parallel / workers);
```

Listing 127: Tutorial extraction: Amdahl speedup limit

Step-by-step reasoning

1. Identify the non-parallelisable fraction.
2. Divide only the parallel part by workers.
3. Add the serial part back.
4. Invert the effective runtime fraction.

Common mistakes

- Dividing the whole program by worker count.
- Forgetting overheads make real speedup lower.
- Treating the model as a guarantee.

Takeaway Adding workers cannot speed up work that remains serial.

10.1.7 Exam-readiness prompts

- Before answering, identify the object or resource whose lifetime is being tested.
- State the API success and failure return values before using the result.
- Draw the relevant storage, descriptor table, process tree, or thread-shared invariant.
- Name the common trap in the question and show how the code avoids it.
- Finish with the cleanup or synchronisation step that makes the example complete.

10.1.8 Depth takeaway

This expansion is intentionally slower than a checklist. The final exam rewards the student who can explain why a line is correct, what would break if one return value or close call were missing, and how the same pattern reappears in a later week.

11 Week 11: Thread Synchronisation and Further Synchronisation

11.1 Core idea

Threads are multiple execution contexts inside one process. They share the process address space, including globals and heap memory, but each has its own stack, registers, thread ID, and program counter. This makes communication easy and races easy.

11.2 Parallelism motivation

The course motivation is the move from relying on faster single cores to using multicore parallelism. Programs must be redesigned to expose independent work. Two common forms are task parallelism, where different tasks run concurrently, and data parallelism, where the same operation is applied to different chunks of data.

11.3 POSIX threads basics

`pthread_t`

- **Context/header:** `##include <pthread.h>`.
- **Form:** `pthread_t tid;`
- **Purpose:** Opaque type identifying a POSIX thread.
- **Result/trap:** Do not assume it is an integer. Store it and pass it to pthread APIs.

`pthread_create`

- **Context/header:** `##include <pthread.h>`; compile/link with `-pthread`.
- **Form:** `int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine)(void *), void *arg);`
- **Purpose:** Creates a new thread that begins by calling `start_routine(arg)`.
- **Result/trap:** Returns 0 on success or an error number. The start function must match `void (*)(void *)`.

`pthread_join`

- **Context/header:** `##include <pthread.h>`.
- **Form:** `int pthread_join(pthread_t thread, void **retval);`
- **Purpose:** Blocks until a thread terminates and optionally collects its return value.
- **Result/trap:** Returns 0 on success or an error number. Joining also prevents leaking thread resources.

```
1 #include <pthread.h>
2 #include <stdio.h>
3
4 void *worker(void *arg) {
5     int id = *(int *)arg;
6     printf("hello from %d\n", id);
7     return NULL;
8 }
9
```

```

10 int main(void) {
11     pthread_t tids[4];
12     int ids[4] = {0, 1, 2, 3};
13
14     for (int i = 0; i < 4; i++) {
15         int rc = pthread_create(&tids[i], NULL, worker, &ids[i]);
16         if (rc != 0) {
17             fprintf(stderr, "pthread_create failed: %d\n", rc);
18         }
19     }
20
21     for (int i = 0; i < 4; i++) {
22         pthread_join(tids[i], NULL);
23     }
24 }

```

Listing 128: Create and join threads

11.4 Thread argument passing

The argument pointer must point to storage that remains valid until the thread has read it. Passing `&i` from the loop variable to every thread is a classic bug because all threads share the same changing variable.

11.5 Race conditions and mutexes

A race condition occurs when behaviour depends on an uncontrolled interleaving of operations. Shared counters and bank-account examples show that read-modify-write sequences are not automatically atomic.

`pthread_mutex_t`

- **Context/header:** `##include <pthread.h>`.
- **Form:** `pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;`
- **Purpose:** Mutex object used for mutual exclusion.
- **Result/trap:** Protect every access to the shared invariant with the same mutex. A mutex has an owner; unlock it in the same logical path that locked it.

`pthread_mutex_lock`, `pthread_mutex_unlock`

- **Context/header:** `##include <pthread.h>`.
- **Form:** `int pthread_mutex_lock(pthread_mutex_t *m); int pthread_mutex_unlock(pthread_mutex_t *m);`
- **Purpose:** Enter and leave a critical section.
- **Result/trap:** Return 0 on success or an error number. Keep critical sections as small as correctness allows.

```

1 static int counter = 0;
2 static pthread_mutex_t counter_lock = PTHREAD_MUTEX_INITIALIZER;
3
4 void *increment_many(void *arg) {
5     int n = *(int *)arg;

```

```

6   for (int i = 0; i < n; i++) {
7       pthread_mutex_lock(&counter_lock);
8       counter++;
9       pthread_mutex_unlock(&counter_lock);
10  }
11  return NULL;
12 }

```

Listing 129: Fix shared counter race with a mutex

11.6 Semaphores

`sem_t`

- **Context/header:** `##include <semaphore.h>`.
- **Form:** `sem_t s;`
- **Purpose:** Semaphore object containing a count.
- **Result/trap:** Unlike a mutex, a semaphore is a counter and does not have the same ownership model.

`sem_wait`, `sem_post`

- **Context/header:** `##include <semaphore.h>`.
- **Form:** `int sem_wait(sem_t *s); int sem_post(sem_t *s);`
- **Purpose:** `sem_wait` waits until the count is positive then decrements; `sem_post` increments and may wake a waiter.
- **Result/trap:** Use semaphores to limit concurrency or order events. Initialise and destroy according to the platform API.

```

1  sem_t ready;
2
3  void *second(void *arg) {
4      sem_wait(&ready);
5      puts("this comes second");
6      return NULL;
7  }
8
9  void *first(void *arg) {
10     puts("this comes first");
11     sem_post(&ready);
12     return NULL;
13 }

```

Listing 130: Semaphore ordering idea

11.7 Condition variables

`pthread_cond_t`

- **Context/header:** `##include <pthread.h>`.
- **Form:** `pthread_cond_t cond = PTHREAD_COND_INITIALIZER;`
- **Purpose:** Condition variable used to wait for a predicate about shared state.

- **Result/trap:** Always pair with a mutex and a predicate variable. The condition variable is not the state itself.

pthread_cond_wait

- **Context/header:** `##include <pthread.h>`.
- **Form:** `int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex);`
- **Purpose:** Atomically unlocks the mutex and waits; when awakened, re-locks the mutex before returning.
- **Result/trap:** Use a `while` loop around it because wakeups can be spurious or another thread may consume the condition first.

pthread_cond_signal

- **Context/header:** `##include <pthread.h>`.
- **Form:** `int pthread_cond_signal(pthread_cond_t *cond);`
- **Purpose:** Wakes at least one thread waiting on the condition variable.
- **Result/trap:** Signal after changing the predicate under the mutex. Use broadcast when many waiters may need to re-check.

```

1 static pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
2 static pthread_cond_t data_ready = PTHREAD_COND_INITIALIZER;
3 static int available = 0;
4 static int item;
5
6 void produce(int x) {
7     pthread_mutex_lock(&m);
8     item = x;
9     available = 1;
10    pthread_cond_signal(&data_ready);
11    pthread_mutex_unlock(&m);
12 }
13
14 int consume(void) {
15     pthread_mutex_lock(&m);
16     while (!available) {
17         pthread_cond_wait(&data_ready, &m);
18     }
19     int x = item;
20     available = 0;
21     pthread_mutex_unlock(&m);
22     return x;
23 }

```

Listing 131: Producer-consumer condition variable pattern

11.8 Barriers

barriers

- **Context/header:** `##include <pthread.h>` where supported.
- **Form:** `pthread_barrier_t b; pthread_barrier_wait(&b);`
- **Purpose:** A barrier is a synchronisation point where a fixed number of threads must arrive before any proceed.

- **Result/trap:** Initialise with the participant count. Destroy when finished. Barriers are useful for generation-style algorithms.

```

1 pthread_barrier_t barrier;
2
3 void *step_worker(void *arg) {
4     for (int gen = 0; gen < MAX_GENERATIONS; gen++) {
5         /* compute this generation's local part */
6         pthread_barrier_wait(&barrier);
7     }
8     return NULL;
9 }

```

Listing 132: Barrier shape

11.9 Deadlock, livelock, starvation

- Deadlock: threads wait forever for resources held by each other.
- Livelock: threads keep changing state but no useful progress occurs.
- Starvation: one thread repeatedly fails to get access while others progress.

Lock ordering is the most reusable prevention pattern: if all code acquires locks in the same global order, circular wait becomes much less likely.

11.10 Amdahl's Law

Amdahl's Law estimates the maximum speedup from parallelisation when some fraction of work remains serial.

$$S(N) = \frac{1}{(1 - P) + \frac{P}{N}}$$

Here P is the parallelisable fraction and N is the number of processors/threads. If 80% of work is parallelisable and 20% is serial, the speedup with 4 workers is:

$$S(4) = \frac{1}{0.2 + \frac{0.8}{4}} = \frac{1}{0.4} = 2.5$$

The exam lesson is that adding threads cannot speed up the serial part. Synchronisation overhead can make real speedup smaller.

Common pitfalls

- Passing the address of a loop variable to every thread.
- Assuming `counter++` is atomic.
- Locking one path but forgetting another path that accesses the same data.
- Using `if` instead of `while` around `pthread_cond_wait`.
- Unlocking a mutex too early and breaking the protected invariant.
- Expecting parallelism to beat Amdahl's Law.

11.11 Tutorial Extractions

11.11.1 Thread argument passing

```
1 pthread_t threads[4];
2 int nums[4] = {1, 2, 3, 4};
3
4 for (int i = 0; i < 4; i++) {
5     pthread_create(&threads[i], NULL, routine, &nums[i]);
6 }
```

Listing 133: Correct per-thread argument storage

Each thread receives a pointer to a distinct stable array element. Passing `&i` would make all threads race to read the same loop variable.

11.11.2 Producer-consumer condition

The producer-consumer tutorial teaches that waiting should be tied to a predicate, not just a signal. The consumer waits while the queue is empty. The producer inserts data, updates the predicate, then signals.

11.11.3 Amdahl calculation

If 95% of a program can run in parallel, the theoretical maximum speedup as N approaches infinity is:

$$\lim_{N \rightarrow \infty} \frac{1}{0.05 + 0.95/N} = 20$$

So even infinite processors cannot beat the 5% serial bottleneck.

Week takeaway

- Threads share memory, which makes communication easy and races dangerous.
- Use mutexes for mutual exclusion, semaphores for counts/events, condition variables for predicates, and barriers for phase boundaries.
- Correct thread code is about invariants and interleavings, not only API calls.
- Amdahl's Law limits speedup when part of the work is serial.

11.12 Full-depth expansion pass

The expanded Week 11 chapter should be the synchronisation chapter the original style implies: every primitive is tied to a predicate or invariant, and every code listing explains what correctness property is being protected.

11.12.1 Teaching-density checklist

Read this week using the original notes standard: each item below should be explainable in words, recognisable in code, and connectable to a failure mode. If an item feels like a vocabulary word only, rewrite it as a small memory, process, descriptor, or thread story.

- interleavings
- race conditions
- critical sections
- mutex ownership
- trylock
- semaphore counts
- condition predicates
- spurious wakeups
- barrier phases
- deadlock cycles
- livelock
- starvation
- lock granularity
- contention

11.12.2 Expanded API cards

Expanded API card: `pthread_mutex_lock`

Purpose Acquire a mutex before entering a critical section.

Header `pthread.h`

Simplified prototype
`int pthread_mutex_lock(pthread_mutex_t *mutex);`

Parameter meanings
 mutex points to the lock protecting the shared invariant.

Return value Zero on success, otherwise an error number.

Failure behaviour
 Can block until the mutex becomes available.

Trap The same mutex must protect all accesses to the invariant.

```

1 pthread_mutex_lock(&lock);
2 shared++;
3 pthread_mutex_unlock(&lock);

```

Listing 134: Small example for `pthread_mutex_lock`

Expanded API card: `pthread_cond_wait`

Purpose Sleep until a condition may have changed, while atomically releasing the mutex.

Header `pthread.h`

Simplified prototype
`int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex);`

Parameter meanings

cond is wait queue; mutex protects the predicate and must be locked before the call.

Return value Zero on success, otherwise an error number; mutex is locked again on return.

Failure behaviour

Can wake when predicate is still false.

Trap

Use while, not if, around the predicate.

```

1 pthread_mutex_lock(&lock);
2 while (count == 0) pthread_cond_wait(&not_empty, &lock);
3 take_item();
4 pthread_mutex_unlock(&lock);

```

Listing 135: Small example for pthread_cond_wait

Expanded API card: pthread_cond_signal

Purpose Wake at least one waiter so it can re-check a predicate.

Header pthread.h

Simplified prototype

```
int pthread_cond_signal(pthread_cond_t *cond);
```

Parameter meanings

cond is the condition variable to signal.

Return value Zero on success, otherwise an error number.

Failure behaviour

Signal is not remembered if no one is waiting.

Trap

Change the predicate under the mutex, then signal.

```

1 pthread_mutex_lock(&lock);
2 count++;
3 pthread_cond_signal(&not_empty);
4 pthread_mutex_unlock(&lock);

```

Listing 136: Small example for pthread_cond_signal

Expanded API card: sem_wait

Purpose Wait until semaphore count is positive, then decrement it.

Header semaphore.h

Simplified prototype

```
int sem_wait(sem_t *sem);
```

Parameter meanings

sem points to an initialised semaphore.

Return value Zero on success, minus one on failure.

Failure behaviour

Can be interrupted by a signal.

Trap

A semaphore is a count, not a mutex with ownership.

```
1 sem_wait(&slots);
2 use_slot();
3 sem_post(&slots);
```

Listing 137: Small example for `sem_wait`

Expanded API card: `pthread_barrier_wait`

Purpose

Block until the configured number of threads reach the barrier.

Header

`pthread.h`

Simplified prototype

`int pthread_barrier_wait(pthread_barrier_t *barrier);`

Parameter meanings

`barrier` points to an initialised barrier object.

Return value

Zero for most threads; one thread may receive `PTHREAD_BARRIER_SERIAL_THREAD`.

Failure behaviour

If too few threads arrive, waiters block forever.

Trap

The barrier count must match actual participants in every phase.

```
1 compute_phase_one();
2 pthread_barrier_wait(&barrier);
3 compute_phase_two();
```

Listing 138: Small example for `pthread_barrier_wait`

11.12.3 Tutorial extractions with full reasoning

11.12.4 Race on counter

What the question is testing

The task checks why increment is a read-modify-write sequence, not an atomic action.

Required background

You need shared state, interleaving, and a mutex around the whole invariant update.

```
1 pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
2 long counter;
3 void *worker(void *arg) {
4     for (int i = 0; i < 100000; i++) {
5         pthread_mutex_lock(&lock);
6         counter++;
7         pthread_mutex_unlock(&lock);
```

```

8     }
9     return NULL;
10 }

```

Listing 139: Tutorial extraction: Race on counter

Step-by-step reasoning

1. counter++ reads the old value.
2. It computes a new value.
3. It writes the new value.
4. Another thread can interleave between these conceptual steps.
5. The mutex serialises the entire update.

Common mistakes

- Locking only the write.
- Using different locks for reads and writes.
- Trusting that a test run with the right answer proves safety.

Takeaway Protect the invariant-sized operation, not merely one line of code.

11.12.5 Producer-consumer condition

What the question is testing The task checks why condition variables are paired with predicates.

Required background You need a mutex, queue count, while-loop wait, and signal after state change.

```

1 pthread_mutex_lock(&q->lock);
2 while (q->count == 0) {
3     pthread_cond_wait(&q->not_empty, &q->lock);
4 }
5 int item = remove_item(q);
6 pthread_cond_signal(&q->not_full);
7 pthread_mutex_unlock(&q->lock);

```

Listing 140: Tutorial extraction: Producer-consumer condition

Step-by-step reasoning

1. Lock before reading the predicate.
2. Wait in a while loop because wakeup means maybe true.
3. pthread_cond_wait releases the mutex while sleeping.
4. On return it holds the mutex again.
5. After removing an item, signal that space may be available.

Common mistakes

- Using if instead of while.
- Waiting without holding the mutex.
- Signalling without changing the predicate.

Takeaway The condition variable is not the condition; the shared predicate is the condition.

11.12.6 Deadlock by lock order

What the question is testing The task checks circular wait.

Required background You need two locks and opposite acquisition orders.

```
1 /* thread A */
2 pthread_mutex_lock(&left);
3 pthread_mutex_lock(&right);
4
5 /* thread B */
6 pthread_mutex_lock(&right);
7 pthread_mutex_lock(&left);
```

Listing 141: Tutorial extraction: Deadlock by lock order

Step-by-step reasoning

1. Thread A can hold left.
2. Thread B can hold right.
3. A waits for right while holding left.
4. B waits for left while holding right.
5. Neither can progress.

Common mistakes

- Adding sleeps instead of fixing ordering.
- Using trylock loops that livelock.
- Forgetting rare schedules still count.

Takeaway Define a global lock order and obey it everywhere.

11.12.7 Barrier phase split

What the question is testing The task checks phase synchronisation.

Required background You need a barrier count equal to the number of participating threads.

```
1 void *worker(void *arg) {
2     phase_one();
3     pthread_barrier_wait(&barrier);
4     phase_two();
```

```

5     return NULL;
6 }

```

Listing 142: Tutorial extraction: Barrier phase split

Step-by-step reasoning

1. All workers run phase one independently.
2. Each blocks at the barrier after phase one.
3. The last arriving worker releases the group.
4. Only then can phase two begin.

Common mistakes

- Setting the count to include main when main does not wait.
- Returning early before the barrier.
- Destroying the barrier before all workers finish.

Takeaway A barrier is a phase promise made by a fixed group of threads.

11.12.8 Exam-readiness prompts

- Before answering, identify the object or resource whose lifetime is being tested.
- State the API success and failure return values before using the result.
- Draw the relevant storage, descriptor table, process tree, or thread-shared invariant.
- Name the common trap in the question and show how the code avoids it.
- Finish with the cleanup or synchronisation step that makes the example complete.

11.12.9 Depth takeaway

This expansion is intentionally slower than a checklist. The final exam rewards the student who can explain why a line is correct, what would break if one return value or close call were missing, and how the same pattern reappears in a later week.

12 Final Revision Patterns and Exam Traps

12.1 Patterns to recognise

- **Address/value/dereference:** distinguish `x`, `&x`, `p`, `*p`, `pp`, and `**pp`.
- **Array plus length:** never rely on a pointer to remember array size.
- **Null-terminated string:** string functions scan until `'\\0'`.
- **Checked resource acquisition:** every `fopen`, `malloc`, `open`, `pipe`, `fork`, `mmap`, and `pthread` call needs a failure path.
- **Own/borrow distinction:** only free memory and close descriptors you own.
- **Fork branch:** handle `-1`, `0`, and positive PID separately.
- **Exec failure path:** code after `exec` is error handling.
- **Pipe closure:** close unused ends immediately.

- **Partial I/O:** robust `read/write` code loops.
- **Thread predicate:** condition variables wait in `while`, not `if`.

12.2 High-value exam traps

- `sizeof` array versus `sizeof` pointer.
- `strncpy` not null-terminating.
- `realloc` failure losing the only pointer.
- `fork` multiplying output counts.
- A pipe read never seeing EOF because some process still has a write end open.
- `waitpid` status macros used in the wrong order.
- `errno` inspected without a failing return value.
- Blocking `read` used in an event loop.
- Passing stack addresses to threads after the stack variable changes or disappears.
- Assuming mutexes make an algorithm faster; they make critical sections safe.

12.3 Function / Macro / Type / Tooling Coverage Checklist

Extracted item	Source lecture/tutorial	Covered in section	Notes
<code>printf</code>	Week 1 lecture/notes	Week 1 Basic I/O	Format string and return value explained.
<code>scanf</code>	Week 1 lecture/notes	Week 1 Basic I/O	Address arguments and checked return.
<code>getchar</code> , <code>putchar</code>	Week 1, Week 3	Weeks 1 and 3	Character stream pattern.
<code>fgets</code>	Tutorial input patterns	Week 2 Tutorial Extraction	Bounded line input.
<code>fscanf</code> , <code>fprintf</code>	Week 4 files	Weeks 1 and 4	Stream formatting and return checks.
<code>fopen</code> , <code>fclose</code>	Week 4 files	Week 4 Streams	Modes, failure, close semantics.
<code>feof</code> , <code>ferror</code> , <code>fflush</code>	Week 4 files/supporting stdio	Week 4 Streams	Correct read loop and buffering.
<code>FILE *</code> , <code>stdin</code> , <code>stdout</code> , <code>stderr</code>	Week 4 and low-level I/O comparison	Weeks 1, 4, 6, 9	Stream versus descriptor distinction.
<code>strcpy</code> , <code>strncpy</code>	Week 3 strings	Week 3 Copying strings	Capacity and termination traps.
<code>strlen</code> , <code>strnlen</code>	Week 3 strings	Week 3 Length	Null termination and bounded length.
<code>strcmp</code> , <code>strncmp</code>	Week 3 strings	Week 3 Comparing strings	Return meaning and prefix comparison.
<code>toupper</code>	Week 3 tutorial/notes	Week 3 Character transform	Requires <code>int</code> character handling.
<code>malloc</code> , <code>calloc</code> , <code>realloc</code> , <code>free</code>	Week 4 memory / Week 5 notes	Week 5 Allocation APIs	Purpose, failure, traps, examples.
<code>sizeof</code> , <code>NULL</code>	Pointers and memory	Weeks 2 and 5	Pointer/array distinction and null sentinel.

Extracted item	Source lecture/tutorial	Covered in section	Notes
gcc, gcc -c, object linking	Week 1 and Week 7 materials	Weeks 1 and 7	Compile, object files, linker.
-Wall, -Wextra, -Werror, -g, -O0, -fsanitize=address	Tooling and dynamic memory debugging	Weeks 1 and 5	Debug and warning workflows.
gdb, valgrind	Tooling notes	Week 5 Memory debugging	Dynamic memory diagnostics.
man 1, man 2, man 3	Week 1 and later API lookups	Week 1 Manual pages	Command/system/library sections.
typedef, enum, union, bitfields	Week 3 lecture / Week 4 notes	Week 4 Data modelling	Data modelling and layout.
struct tm, struct stat	Week 4 files/time and IPC/mmap support	Weeks 4 and 10	Time fields and file metadata.
Function pointer syntax, qsort	Week 5 lecture / Week 6 tutorial	Week 6 Function pointers	Comparator pattern included.
signal, sigaction	Week 5 lecture / Week 6 tutorial	Week 6 Signals	Preferred sigaction, flag handler.
errno, perror, strerror, EINTR	Week 5 lecture	Week 6 errno and I/O	Failure and interrupted calls.
open, read, write, close, file descriptors	Week 5 lecture / Week 6 tutorial	Week 6 Low-level I/O	Return values, partial transfer, blocking.
fcntl, O_NONBLOCK	Week 6 tutorial / IPC lecture	Weeks 6 and 10	Non-blocking setup.
fork	Week 7 lecture / Week 8 tutorial	Week 8 Processes	Return cases and output counting.
exec, execl, execlp, execv, execvp	Week 7 lecture / Week 8 tutorial	Week 8 exec family	Replacement semantics.
posix_spawn	Week 7 lecture / Week 8 IPC	Weeks 8 and 9	Spawn and file actions.
wait, waitpid, WIFEXITED, WEXITSTATUS	Week 8 tutorial	Week 8 Waiting	Status macros and zombies.
Exit status, zombies, fork bombs	Week 8 tutorial	Week 8 Waiting and traps	Process table cleanup and unsafe fork loops.
pipe, pipe EOF, closing unused ends	Week 8 IPC / Week 9 tutorial	Week 9 Pipes	One-way channel and EOF.
dup, dup2, redirection	Week 9 tutorial	Week 9 Redirection	Descriptor table remapping.
fdopen, fileno	Week 9 tutorial	Week 9 fd/stream bridge	Stream wrapper and descriptor extraction.
popen, pclose	Week 8 tutorial	Week 9 popen/pclose	Command streams and close status.
mmap, munmap, MAP_SHARED, MAP_PRIVATE, PROT_READ, PROT_WRITE	Week 8 IPC / Week 9 tutorial	Week 9 IPC extension	Mapping purpose, permissions, cleanup.

Extracted item	Source lecture/tutorial	Covered in section	Notes
<code>select</code> , <code>FD_ZERO</code> , <code>FD_SET</code> , <code>FD_ISSET</code>	Week 8 IPC	Week 10 select	Descriptor readiness sets.
<code>poll</code> , <code>epoll</code>	Week 9 IPC tutorial	Week 10 poll/epoll	Multi-descriptor readiness.
<code>pthread_create</code> , <code>pthread_join</code> , <code>pthread_t</code>	Week 9 threads / Week 10 tutorial	Week 10 pthread basics	Creation, join, argument passing.
<code>pthread_mutex_t</code> , <code>pthread_mutex_lock</code> , <code>pthread_mutex_unlock</code>	Week 10 synchronisation	Week 11 Mutexes	Critical sections and races.
<code>pthread_cond_t</code> , <code>pthread_cond_wait</code> , <code>pthread_cond_signal</code>	Week 11a further sync	Week 11 Condition variables	Predicate and while-loop pattern.
<code>sem_t</code> , <code>sem_wait</code> , <code>sem_post</code>	Week 10 synchronisation	Week 11 Semaphores	Counter/event synchronisation.
Barriers	Week 11a further sync	Week 11 Barriers	Phase synchronisation.
Amdahl's Law	Week 10 synchronisation	Week 10 Amdahl's Law	Formula and example.